CourseworkMA - combined Java sources
Total files: 36

================================================================
FILE: app\src\androidTest\java\com\example\treasurehuntapplication\ExampleInstrumentedTest.java
================================================================

```
package com.example.treasurehuntapplication;

import android.content.Context;

import androidx.test.platform.app.InstrumentationRegistry;
import androidx.test.ext.junit.runners.AndroidJUnit4;

import org.junit.Test;
import org.junit.runner.RunWith;

import static org.junit.Assert.*;

/**
 * Instrumented test, which will execute on an Android device.
 *
 * @see <a href="http://d.android.com/tools/testing">Testing documentation</a>
 */
@RunWith(AndroidJUnit4.class)
public class ExampleInstrumentedTest {
    @Test
    public void useAppContext() {
        // Context of the app under test.
        Context appContext =
InstrumentationRegistry.getInstrumentation().getTargetContext();
        assertEquals("com.example.treasurehuntapplication",
appContext.getPackageName());
    }
}
```

================================================================

FILE:
app\src\main\java\com\example\treasurehuntapplication\ActiveTreasureManager.java
==========

```
package com.example.treasurehuntapplication;

import android.content.Context;
import android.content.SharedPreferences;

/**
 * Tracks which treasure the user is currently hunting (after accepting a clue on the treasure screen).
 * Stored per signed-in user so accounts do not share hunt state on the same device.
 */
public final class ActiveTreasureManager {

    // --- Sentinel & preference namespace ---
    private static final int NONE = -1;
    private static final String PREF = "active_treasure";
    private static final String KEY_PREFIX = "active_id_";

    private ActiveTreasureManager() {
    }

    // --- Key per user (email-based) ---

    private static SharedPreferences prefs(Context context) {
        return context.getApplicationContext().getSharedPreferences(PREF, Context.MODE_PRIVATE);
    }

    private static String safeKey(String raw) {
        return raw.trim().toLowerCase().replaceAll("[^a-z0-9]+", "_");
    }

    private static String keyForUser(Context context) {
        String email = UserSession.getEmail(context.getApplicationContext());
        if (email == null || email.isEmpty()) {
            return KEY_PREFIX + "guest";
        }
        return KEY_PREFIX + safeKey(email);
    }
```

```
    // --- Active hunt id ---

    public static void setActiveTreasure(Context context, int id) {
        prefs(context).edit().putInt(keyForUser(context), id).apply();
    }

    public static void clearActiveTreasure(Context context) {
        prefs(context).edit().remove(keyForUser(context)).apply();
    }

    public static int getActiveTreasureId(Context context) {
        String k = keyForUser(context);
        if (!prefs(context).contains(k)) {
            return NONE;
        }
        return prefs(context).getInt(k, NONE);
    }

    public static boolean hasActiveTreasure(Context context) {
        return getActiveTreasureId(context) != NONE;
    }

    public static Treasure getActiveTreasure(Context context) {
        int id = getActiveTreasureId(context);
        if (id == NONE) {
            return null;
        }
        return TreasureManager.getById(id);
    }

    public static boolean isActive(Context context, int id) {
        return getActiveTreasureId(context) == id;
    }

    // --- Reset (all users on device) ---

    /** Clears active-hunt state for every account on this device. */
    public static void clearAllActiveTreasureState(Context context) {
        prefs(context).edit().clear().apply();
    }
}
```

```
================================================================================
FILE:
app\src\main\java\com\example\treasurehuntapplication\AuthInputValidator.java
================================================================================

package com.example.treasurehuntapplication;

import android.util.Patterns;

/**
 * Normalizes and validates login and signup fields.
 * <p>
 * Firebase email flows require Nottingham Trent University student addresses: the substring after
 * the final {@code @} must equal {@link #REQUIRED_EMAIL_DOMAIN} (personal domains such as
 * {@code gmail.com} or {@code outlook.com}, staff domains like {@code ntu.ac.uk} without {@code my.},
 * and subdomains such as {@code mail.my.ntu.ac.uk} are rejected).
 */
public final class AuthInputValidator {

    public static final int MIN_PASSWORD_LENGTH = 6;

    /**
     * Host part (domain) that {@link #isAllowedNtuLoginEmail} requires after the last {@code @}.
     * Compares against {@link #normalizeEmail(String)} output, so matching is case-insensitive.
     */
    public static final String REQUIRED_EMAIL_DOMAIN = "my.ntu.ac.uk";

    private AuthInputValidator() {
    }

    public static String normalizeEmail(String raw) {
        if (raw == null) {
            return "";
        }
        return raw.trim().toLowerCase();
    }

    public static boolean isValidEmail(String email) {
        return email != null && !email.isEmpty() &&
```

```
Patterns.EMAIL_ADDRESS.matcher(email).matches();
    }

   /**
     * Whether {@code email} may be used for Firebase sign-in, password reset, or
registration.
     * <ul>
     *   <li>Must pass {@link #isValidEmail(String)} (Android {@link
android.util.Patterns#EMAIL_ADDRESS}).</li>
     *   <li>After {@link #normalizeEmail(String)}, the substring after the
<em>last</em> {@code @} must equal
     *      {@link #REQUIRED_EMAIL_DOMAIN} exactly (case-insensitive because of
normalization).</li>
     * </ul>
     * Examples: {@code alice@my.ntu.ac.uk} and {@code Bob@MY.NTU.AC.UK}
allowed;
     * {@code x@gmail.com}, {@code x@outlook.com}, {@code x@ntu.ac.uk} (no
{@code my.}), and
     * {@code x@mail.my.ntu.ac.uk} (subdomain, not exact host) are not allowed.
     * {@code lastIndexOf('@')} handles local parts that themselves contain {@code
@}.
     */
    public static boolean isAllowedNtuLoginEmail(String email) {
        if (!isValidEmail(email)) {
            return false;
        }
        String e = normalizeEmail(email);
        int at = e.lastIndexOf('@');
        if (at < 1) {
            return false;
        }
        return REQUIRED_EMAIL_DOMAIN.equals(e.substring(at + 1));
    }

   public static boolean isValidPasswordLength(String password) {
        return password != null && password.length() >= MIN_PASSWORD_LENGTH;
    }
}
```

```
==================================================
====================
```

FILE: app\src\main\java\com\example\treasurehuntapplication\CampusMapDefaults.java

```
================================================================
====================

package com.example.treasurehuntapplication;

import android.location.Location;

import com.google.android.gms.maps.model.LatLng;

/** Default map camera for NTU Clifton area. */
 public final class CampusMapDefaults {

    public static final float ZOOM_OVERVIEW = 16f;

    private static final LatLng CLIFTON_CAMPUS = new LatLng(52.9115, -1.1865);
     /** Rough radius (m) within which we prefer the device location over the
campus default. */
     private static final float NEAR_CAMPUS_METERS = 25_000f;

    private CampusMapDefaults() {
     }

    public static LatLng cliftonCampus() {
        return CLIFTON_CAMPUS;
     }

    public static boolean shouldUseDeviceLocationForCamera(Location location) {
        if (location == null) {
            return false;
        }
        float[] out = new float[1];
        Location.distanceBetween(
                location.getLatitude(),
                location.getLongitude(),
                CLIFTON_CAMPUS.latitude,
                CLIFTON_CAMPUS.longitude,
                out);
        return out[0] <= NEAR_CAMPUS_METERS;
    }
 }



================================================================
====================
```

FILE: app\src\main\java\com\example\treasurehuntapplication\DevOfflineLogin.java
==========

```
package com.example.treasurehuntapplication;

/**
  * Optional extra offline credentials for development. No pairs are enabled by
default;
  * extend {@link #matches(String, String)} locally if you need demo accounts
beyond {@code a}/{@code a}.
  */
 public final class DevOfflineLogin {

    public static final String OFFLINE_SESSION_EMAIL = "offline.dev@ntu.local";

    private DevOfflineLogin() {
     }

    public static boolean matches(String rawId, String password) {
        return false;
     }
 }
```

==========
FILE:
app\src\main\java\com\example\treasurehuntapplication\FirestoreUserDoc.java
==========

```
package com.example.treasurehuntapplication;

import android.content.Context;

import androidx.annotation.Nullable;

import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;

/**
 * Single place for {@code users/{documentId}} ids so profile, leaderboard sync,
and game state
```

```
 * always target the same Firestore document (normalized email).
 */
public final class FirestoreUserDoc {

    private FirestoreUserDoc() {
    }

    /** Document id from a Firebase user, or null if missing. */
    @Nullable
    public static String idForFirebaseUser(@Nullable FirebaseUser u) {
        if (u == null || u.getEmail() == null || u.getEmail().isEmpty()) {
            return null;
        }
        return AuthInputValidator.normalizeEmail(u.getEmail());
    }

    /** Document id from {@link UserSession}, or null if not signed in. */
    @Nullable
    public static String idForSession(Context context) {
        String email = UserSession.getEmail(context.getApplicationContext());
        if (email == null || email.isEmpty()) {
            return null;
        }
        return AuthInputValidator.normalizeEmail(email);
    }

    /**
     * Prefer the signed-in Firebase accountâ€™s email, else session email (e.g.
offline flows).
     */
    @Nullable
    public static String idForCurrentUser(Context context) {
        String fromAuth =
idForFirebaseUser(FirebaseAuth.getInstance().getCurrentUser());
        if (fromAuth != null) {
            return fromAuth;
        }
        return idForSession(context);
    }
}
```

==========
==========

FILE:
app\src\main\java\com\example\treasurehuntapplication\GameStateReset.java
==========
==========

```
package com.example.treasurehuntapplication;

import android.content.Context;

/** Clears local hunt progress for shipped data migrations. */
 public final class GameStateReset {

    private GameStateReset() {
     }

    public static void clearLocalGameProgress(Context context) {
        Context app = context.getApplicationContext();
        TreasureManager.clearAllCollectedState(app);
        ActiveTreasureManager.clearAllActiveTreasureState(app);
        WalkDistanceTracker.clearAll(app);
        ScoreRepository.clearAll(app);
    }
 }
```

==========
==========
FILE:
app\src\main\java\com\example\treasurehuntapplication\HexagonImageView.java
==========
==========

```
package com.example.treasurehuntapplication;

import android.content.Context;
 import android.graphics.Canvas;
 import android.graphics.Color;
 import android.graphics.Paint;
 import android.graphics.Path;
 import android.os.Build;
 import android.util.AttributeSet;
 import android.view.View;

import androidx.appcompat.widget.AppCompatImageView;
```

```
/**
  * {@link AppCompatImageView} clipped to a regular hexagon, with an optional
stroke border.
  */
 public class HexagonImageView extends AppCompatImageView {

    private final Path hexPath = new Path();
     private float strokeWidthPx;
     private int strokeColor = Color.GRAY;
     private final Paint strokePaint = new Paint(Paint.ANTI_ALIAS_FLAG);

    public HexagonImageView(Context context) {
        super(context);
        init();
     }

    public HexagonImageView(Context context, AttributeSet attrs) {
        super(context, attrs);
        init();
     }

    public HexagonImageView(Context context, AttributeSet attrs, int defStyleAttr) {
        super(context, attrs, defStyleAttr);
        init();
     }

    private void init() {
        setScaleType(ScaleType.CENTER_CROP);
        strokePaint.setStyle(Paint.Style.STROKE);
        if (Build.VERSION.SDK_INT < Build.VERSION_CODES.O) {
            setLayerType(View.LAYER_TYPE_SOFTWARE, null);
        }
     }

    @Override
     protected void onSizeChanged(int w, int h, int oldw, int oldh) {
        super.onSizeChanged(w, h, oldw, oldh);
        buildHexPath(w, h);
     }

    private void buildHexPath(int w, int h) {
        hexPath.reset();
        float cx = w / 2f;
        float cy = h / 2f;
        float inset = Math.max(0f, strokeWidthPx / 2f);
```

```
        float r = Math.min(w, h) / 2f - inset;
        if (r <= 0f) {
            return;
        }
        for (int i = 0; i < 6; i++) {
            double angle = Math.PI / 3 * i - Math.PI / 6;
            float x = (float) (cx + r * Math.cos(angle));
            float y = (float) (cy + r * Math.sin(angle));
            if (i == 0) {
                hexPath.moveTo(x, y);
            } else {
                hexPath.lineTo(x, y);
            }
        }
        hexPath.close();
    }

    public void setStrokeWidthDp(float dp) {
        strokeWidthPx = dp * getResources().getDisplayMetrics().density;
        strokePaint.setStrokeWidth(strokeWidthPx);
        buildHexPath(getWidth(), getHeight());
        invalidate();
    }

    public void setStrokeColor(int color) {
        strokeColor = color;
        strokePaint.setColor(color);
        invalidate();
    }

    @Override
    protected void onDraw(Canvas canvas) {
        if (hexPath.isEmpty()) {
            buildHexPath(getWidth(), getHeight());
        }
        canvas.save();
        canvas.clipPath(hexPath);
        super.onDraw(canvas);
        canvas.restore();
        strokePaint.setColor(strokeColor);
        strokePaint.setStrokeWidth(strokeWidthPx);
        canvas.drawPath(hexPath, strokePaint);
    }
}
```

```
================================================================
====================
FILE:
app\src\main\java\com\example\treasurehuntapplication\LeaderboardAdapter.java
================================================================
====================

package com.example.treasurehuntapplication;

import android.text.TextUtils;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.ImageView;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.recyclerview.widget.RecyclerView;

import java.util.List;

/**
 * Simple list rows for {@link LeaderboardFragment}: rank, display name, score;
click â†’ profile dialog.
 */
public class LeaderboardAdapter extends
RecyclerView.Adapter<LeaderboardAdapter.ViewHolder> {

    public interface OnPlayerClickListener {
        void onPlayerClick(LeaderboardModel user);
    }

    private final List<LeaderboardModel> userList;
    @Nullable
    private OnPlayerClickListener onPlayerClickListener;

    public LeaderboardAdapter(List<LeaderboardModel> userList) {
        this.userList = userList;
    }

    public void setOnPlayerClickListener(@Nullable OnPlayerClickListener listener) {
        this.onPlayerClickListener = listener;
    }
```

```
    // --- RecyclerView.Adapter ---

    @NonNull
    @Override
    public ViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int
viewType) {
        View view = LayoutInflater.from(parent.getContext())
                .inflate(R.layout.leaderboard_row_template, parent, false);
        return new ViewHolder(view);
    }

    @Override
    public void onBindViewHolder(@NonNull ViewHolder holder, int position) {
        LeaderboardModel user = userList.get(position);

        holder.tvRank.setText(String.valueOf(position + 1));
        holder.tvUsername.setText(user.getDisplayName());
        holder.tvScore.setText(user.getScore() + " pts");

        String sub = user.getProfileLocation();
        if (TextUtils.isEmpty(sub)) {
            holder.playerLocation.setText(R.string.leaderboard_row_campus_hint);
        } else {
            holder.playerLocation.setText(sub);
        }

        holder.playerImage.setImageResource(R.drawable.baseline_person_24);

        holder.itemView.setClickable(true);
        holder.itemView.setFocusable(true);
        holder.itemView.setOnClickListener(v -> {
            if (onPlayerClickListener != null) {
                onPlayerClickListener.onPlayerClick(user);
            }
        });
    }

    @Override
    public int getItemCount() {
        return userList.size();
    }

    /** Row view holder: rank, name, points. */
    public static class ViewHolder extends RecyclerView.ViewHolder {
```

```
        TextView tvRank;
        TextView tvUsername;
        TextView tvScore;
        ImageView playerImage;
        TextView playerLocation;

        public ViewHolder(@NonNull View itemView) {
            super(itemView);
            tvRank = itemView.findViewById(R.id.tvRank);
            tvUsername = itemView.findViewById(R.id.tvUsername);
            tvScore = itemView.findViewById(R.id.tvScore);
            playerImage = itemView.findViewById(R.id.playerImage);
            playerLocation = itemView.findViewById(R.id.playerLocation);
        }
    }
}
```

```
================================================================
====================
FILE:
app\src\main\java\com\example\treasurehuntapplication\LeaderboardFragment.java
================================================================
====================

package com.example.treasurehuntapplication;

import android.os.Bundle;
import android.util.Log;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.google.firebase.firestore.DocumentSnapshot;
import com.google.firebase.firestore.FirebaseFirestore;
import com.google.firebase.firestore.ListenerRegistration;
import com.google.firebase.firestore.QuerySnapshot;
```

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * Live Firestore leaderboard: listens to {@code users} collection, sorts by score,
shows top N,
 * empty state, and opens {@link PublicPlayerStatsDialog} on row tap.
 */
public class LeaderboardFragment extends Fragment {

    private static final String TAG = "LeaderboardFragment";
    private static final int MAX_FETCH = 200;
    private static final int TOP_N = 50;

    private RecyclerView recyclerView;
    private TextView emptyView;
    private LeaderboardAdapter adapter;
    private List<LeaderboardModel> userList;
    private FirebaseFirestore db;
    @Nullable
    private ListenerRegistration leaderboardRegistration;

    public LeaderboardFragment() {
    }

    @Nullable
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup
container, Bundle savedInstanceState) {
        View view = inflater.inflate(R.layout.fragment_leaderboard, container, false);

        // --- RecyclerView + adapter ---
        userList = new ArrayList<>();
        adapter = new LeaderboardAdapter(userList);

        recyclerView = view.findViewById(R.id.leaderboardRecycler);
        emptyView = view.findViewById(R.id.leaderboardEmpty);
        recyclerView.setLayoutManager(new LinearLayoutManager(requireContext()));
        recyclerView.setAdapter(adapter);
        adapter.setOnPlayerClickListener(user ->
PublicPlayerStatsDialog.show(LeaderboardFragment.this, user));

        db = FirebaseFirestore.getInstance();
```

```
        return view;
    }

    @Override
    public void onStart() {
        super.onStart();
        // --- Subscribe to Firestore (active while fragment started) ---
        attachLeaderboardListener();
    }

    @Override
    public void onStop() {
        detachLeaderboardListener();
        super.onStop();
    }

    /** Removes snapshot listener to avoid leaks and wasted reads. */
    private void detachLeaderboardListener() {
        if (leaderboardRegistration != null) {
            leaderboardRegistration.remove();
            leaderboardRegistration = null;
        }
    }

    /** Attaches listener to {@code users} with limit; handles errors and drives
{@link #applyLeaderboardDocuments}. */
    private void attachLeaderboardListener() {
        detachLeaderboardListener();

        leaderboardRegistration = db.collection("users")
                .limit(MAX_FETCH)
                .addSnapshotListener((QuerySnapshot snapshot,
com.google.firebase.firestore.FirebaseFirestoreException error) -> {
                    if (!isAdded()) {
                        return;
                    }
                    if (error != null) {
                        Log.e(TAG, "Leaderboard listener failed", error);
                        if (getContext() != null) {
                            String msg = error.getMessage() != null ? error.getMessage() :
"Unknown error";
                            Toast.makeText(getContext(),
getString(R.string.leaderboard_error, msg), Toast.LENGTH_LONG).show();
                        }
                        updateEmptyState();
```

```
                return;
            }
            if (snapshot == null) {
                return;
            }
            applyLeaderboardDocuments(snapshot);
        });
    }

    /** Parses documents, sorts descending by score, trims to TOP_N. */
    private void applyLeaderboardDocuments(QuerySnapshot snapshot) {
        userList.clear();
        for (DocumentSnapshot document : snapshot.getDocuments()) {
            userList.add(LeaderboardModel.fromDocument(document));
        }

        Collections.sort(userList, (a, b) -> Long.compare(b.getScore(), a.getScore()));
        if (userList.size() > TOP_N) {
            List<LeaderboardModel> top = new ArrayList<>(userList.subList(0,
TOP_N));
            userList.clear();
            userList.addAll(top);
        }

        adapter.notifyDataSetChanged();
        updateEmptyState();
    }

    /** Toggles placeholder vs list when there are zero rows. */
    private void updateEmptyState() {
        if (emptyView == null || recyclerView == null) {
            return;
        }
        boolean empty = userList.isEmpty();
        emptyView.setVisibility(empty ? View.VISIBLE : View.GONE);
        recyclerView.setVisibility(empty ? View.GONE : View.VISIBLE);
    }
}
```

==========
==========
FILE:
app\src\main\java\com\example\treasurehuntapplication\LeaderboardModel.java

```
================================================================
====================
```

```
package com.example.treasurehuntapplication;

import androidx.annotation.NonNull;

import com.google.firebase.firestore.DocumentSnapshot;

/**
 * Firestore user fields used on the leaderboard.
 * Documents use {@code username} (display) and {@code score}; {@code email}
is a fallback for older docs.
 */
public class LeaderboardModel {

    // --- Row fields (from Firestore or local demo) ---
    private String username;
    private String email;
    private long score;
    /** Optional subtitle under the display name (e.g. campus line from profile
settings). */
    private String profileLocation;
    /** Firestore document id (used to load full profile); may differ from email in
edge cases. */
    private String firestoreDocumentId;
    /** Firebase Auth uid when present â€” matches the signed-in user even if email
fields differ. */
    private String firebaseUid;

    public LeaderboardModel() {
    }

    public LeaderboardModel(String username, long score) {
        this.username = username;
        this.score = score;
    }

    // --- Firestore mapping ---

    /**
     * Reads only leaderboard fields. Avoids {@link
DocumentSnapshot#toObject(Class)} so extra
     * document keys (e.g. legacy {@code password}, nested profile maps) do not
spam the mapper.
```

```
     */
    @NonNull
    public static LeaderboardModel fromDocument(@NonNull DocumentSnapshot
document) {
        LeaderboardModel user = new LeaderboardModel();
        if (document.contains("score")) {
            Long s = document.getLong("score");
            user.setScore(s != null ? s : 0L);
        }
        String docEmail = document.getString("email");
        if (docEmail == null || docEmail.isEmpty()) {
            docEmail = document.getId();
        }
        user.setEmail(docEmail);
        String u = document.getString("username");
        user.setUsername(u != null && !u.isEmpty() ? u : docEmail);
        String loc = document.getString("profileLocation");
        user.setProfileLocation(loc != null ? loc : "");
        String uid = document.getString("uid");
        user.setFirebaseUid(uid != null ? uid : "");
        user.setFirestoreDocumentId(document.getId());
        return user;
    }

    // --- Accessors ---

    public String getFirestoreDocumentId() {
        if (firestoreDocumentId != null && !firestoreDocumentId.isEmpty()) {
            return firestoreDocumentId;
        }
        if (email != null && !email.isEmpty()) {
            return email.trim();
        }
        return "";
    }

    public void setFirestoreDocumentId(String firestoreDocumentId) {
        this.firestoreDocumentId = firestoreDocumentId;
    }

    public String getUsername() {
        return username;
    }

    public String getEmail() {
        return email;
```

```
    }

    public long getScore() {
        return score;
    }

    public void setScore(long score) {
        this.score = score;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getProfileLocation() {
        return profileLocation != null ? profileLocation : "";
    }

    public void setProfileLocation(String profileLocation) {
        this.profileLocation = profileLocation != null ? profileLocation : "";
    }

    public String getFirebaseUid() {
        return firebaseUid != null ? firebaseUid : "";
    }

    public void setFirebaseUid(String firebaseUid) {
        this.firebaseUid = firebaseUid != null ? firebaseUid : "";
    }

    /** Prefer username; fall back to email for legacy documents that only stored
email. */
    public String getDisplayName() {
        if (username != null && !username.trim().isEmpty()) {
            return username;
        }
        if (email != null && !email.trim().isEmpty()) {
            return email;
        }
        return "Player";
    }
```

```
}
```

```
================================================================
====================
FILE:
app\src\main\java\com\example\treasurehuntapplication\LeaderboardUserRegistrati
on.java
================================================================
====================

package com.example.treasurehuntapplication;

import android.content.Context;

import com.google.firebase.auth.FirebaseAuth;
 import com.google.firebase.auth.FirebaseUser;
 import com.google.firebase.firestore.FirebaseFirestore;
 import com.google.firebase.firestore.SetOptions;

import java.util.HashMap;
 import java.util.Map;

/** Ensures a {@code users/{email}} row exists for Firebase-signed-in accounts. */
 public final class LeaderboardUserRegistration {

    private LeaderboardUserRegistration() {
     }

    public static void ensureRegistered(Context context) {
        FirebaseUser u = FirebaseAuth.getInstance().getCurrentUser();
        if (u == null || u.getEmail() == null || u.getEmail().isEmpty()) {
            return;
        }
        String docId = FirestoreUserDoc.idForFirebaseUser(u);
        if (docId == null) {
            return;
        }
        FirebaseFirestore.getInstance()
                .collection("users")
                .document(docId)
                .get()
                .addOnSuccessListener(doc -> {
                    if (!doc.exists()) {
```

```
                    Map<String, Object> initial = new HashMap<>();
                    initial.put("uid", u.getUid());
                    initial.put("email", docId);
                    initial.put("username", docId);
                    initial.put("score",
ScoreRepository.getLocalScore(context.getApplicationContext()));
                    FirebaseFirestore.getInstance()
                            .collection("users")
                            .document(docId)
                            .set(initial, SetOptions.merge());
                    return;
                }
                String existingUid = doc.getString("uid");
                if (existingUid == null || existingUid.isEmpty()) {
                    Map<String, Object> patch = new HashMap<>();
                    patch.put("uid", u.getUid());
                    FirebaseFirestore.getInstance()
                            .collection("users")
                            .document(docId)
                            .set(patch, SetOptions.merge());
                }
            });
    }
}
```

==========
==========
FILE: app\src\main\java\com\example\treasurehuntapplication\Login.java
==========
==========

```
package com.example.treasurehuntapplication;

import android.content.Intent;
import android.os.Bundle;
import android.text.InputType;
import android.util.Log;
import android.widget.Button;
import android.widget.CheckBox;
import android.widget.EditText;
import android.widget.TextView;
import android.widget.Toast;

import androidx.activity.EdgeToEdge;
```

```
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

import com.google.firebase.FirebaseNetworkException;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseAuthInvalidCredentialsException;
import com.google.firebase.auth.FirebaseAuthInvalidUserException;
import com.google.firebase.auth.FirebaseAuthUserCollisionException;
import com.google.firebase.firestore.FirebaseFirestore;

/**
 * Sign-in screen: Firebase Auth, optional legacy Firestore password, offline demo logins,
 * remember-me (username only), show-password toggle, and hand-off to {@link navigation}.
 * <p>
 * Email/password and password-reset requests that reach Firebase must use an {@code @my.ntu.ac.uk}
 * address ({@link AuthInputValidator#isAllowedNtuLoginEmail}); offline shortcuts ({@code a}/{@code a},
 * {@code admin}, {@link DevOfflineLogin}) skip that check.
 */
public class Login extends AppCompatActivity {

    private static final String TAG = "Login";

    // --- Firebase & UI ---
    private FirebaseFirestore db;
    private FirebaseAuth auth;
    private EditText usernameInput;
    private EditText passwordInput;
    private CheckBox rememberCheck;
    private Button enterButton;
    private Button signupButton;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_login);

        // --- Edge-to-edge: pad root for system bars / display cutout ---
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, windowInsets) -> {
```

```
            Insets bars =
windowInsets.getInsets(WindowInsetsCompat.Type.systemBars());
            Insets cutout =
windowInsets.getInsets(WindowInsetsCompat.Type.displayCutout());
            v.setPadding(
                    Math.max(bars.left, cutout.left),
                    Math.max(bars.top, cutout.top),
                    Math.max(bars.right, cutout.right),
                    Math.max(bars.bottom, cutout.bottom));
            return windowInsets;
        });

        // --- Services ---
        db = FirebaseFirestore.getInstance();
        auth = FirebaseAuth.getInstance();

        // --- Form views ---
        usernameInput = findViewById(R.id.usernameInput);
        passwordInput = findViewById(R.id.passwordInput);
        rememberCheck = findViewById(R.id.rememberCheck);
        CheckBox showPasswordCheck = findViewById(R.id.showPasswordCheck);
        enterButton = findViewById(R.id.login_enter);
        signupButton = findViewById(R.id.signupButton);

        // --- Remember me: pre-fill username from last session ---
        LoginRememberPreferences.loadIntoForm(this, usernameInput,
rememberCheck);

        // --- Show password (plain text vs hidden) ---
        showPasswordCheck.setOnCheckedChangeListener((buttonView, isChecked) ->
                applyPasswordVisibility(isChecked));

        // --- Submit: empty check â†’ offline "a"/a â†’ dev â†’ email validation â†’
Firebase ---
        enterButton.setOnClickListener(v -> {
            String rawId = usernameInput.getText().toString().trim();
            String password = passwordInput.getText().toString();

            if (rawId.isEmpty() || password.isEmpty()) {
                Toast.makeText(Login.this, R.string.login_empty_fields,
Toast.LENGTH_SHORT).show();
                return;
            }

            if ("a".equalsIgnoreCase(rawId) && "a".equals(password)) {
                UserSession.signIn(Login.this, "a");
```

```
            LoginRememberPreferences.save(Login.this,
rememberCheck.isChecked(), rawId);
            Toast.makeText(Login.this, R.string.offline_login_success,
Toast.LENGTH_SHORT).show();
            goToMain();
            return;
        }

        if ("admin".equalsIgnoreCase(rawId) && "admin123".equals(password)) {
            loginUser(rawId, password, rawId);
            return;
        }

        if (DevOfflineLogin.matches(rawId, password)) {
            UserSession.signIn(Login.this,
DevOfflineLogin.OFFLINE_SESSION_EMAIL);
            LoginRememberPreferences.save(Login.this,
rememberCheck.isChecked(), rawId);
            Toast.makeText(Login.this, R.string.dev_offline_login_success,
Toast.LENGTH_SHORT).show();
            goToMain();
            return;
        }

        String email = AuthInputValidator.normalizeEmail(rawId);
        if (!AuthInputValidator.isValidEmail(email)) {
            Toast.makeText(Login.this, R.string.auth_invalid_email,
Toast.LENGTH_SHORT).show();
            return;
        }
        // Block before Firebase: only NTU student mail host (my.ntu.ac.uk), not
Gmail/Outlook/etc.
        if (!AuthInputValidator.isAllowedNtuLoginEmail(email)) {
            Toast.makeText(Login.this, R.string.auth_email_ntu_only,
Toast.LENGTH_LONG).show();
            return;
        }

        loginUser(email, password, rawId);
    });

    signupButton.setOnClickListener(v ->
            startActivity(new Intent(Login.this, Signup.class)));

    TextView forgotPassword = findViewById(R.id.forgotPassword);
    forgotPassword.setOnClickListener(v -> {
```

```
            String rawId = usernameInput.getText().toString().trim();
            String email = AuthInputValidator.normalizeEmail(rawId);
            if (!AuthInputValidator.isValidEmail(email)) {
                Toast.makeText(Login.this, R.string.auth_invalid_email,
Toast.LENGTH_SHORT).show();
                return;
            }
            // Same domain rule as sign-in so reset links are not sent to non-NTU
inboxes.
            if (!AuthInputValidator.isAllowedNtuLoginEmail(email)) {
                Toast.makeText(Login.this, R.string.auth_email_ntu_only,
Toast.LENGTH_LONG).show();
                return;
            }
            auth.sendPasswordResetEmail(email)
                    .addOnSuccessListener(unused ->
                            Toast.makeText(Login.this, R.string.auth_password_reset_sent,
Toast.LENGTH_LONG).show())
                    .addOnFailureListener(e ->
                            Toast.makeText(Login.this, R.string.auth_password_reset_failed,
Toast.LENGTH_LONG).show());
        });
    }

    private void applyPasswordVisibility(boolean visible) {
        int end = passwordInput.getSelectionEnd();
        if (visible) {
            passwordInput.setInputType(InputType.TYPE_CLASS_TEXT
                    | InputType.TYPE_TEXT_VARIATION_VISIBLE_PASSWORD);
        } else {
            passwordInput.setInputType(InputType.TYPE_CLASS_TEXT
                    | InputType.TYPE_TEXT_VARIATION_PASSWORD);
        }
        int len = passwordInput.length();
        if (end >= 0 && end <= len) {
            passwordInput.setSelection(end);
        } else if (len > 0) {
            passwordInput.setSelection(len);
        }
    }

    private void loginUser(String email, String password, String rememberHint) {
        if ("admin".equalsIgnoreCase(email.trim()) && "admin123".equals(password))
{
            UserSession.signIn(this, email.trim().toLowerCase());
```

```
            LoginRememberPreferences.save(this, rememberCheck.isChecked(),
rememberHint);
            Toast.makeText(this, R.string.offline_login_success,
Toast.LENGTH_SHORT).show();
            goToMain();
            return;
        }

        auth.signInWithEmailAndPassword(email, password)
                .addOnSuccessListener(result -> {
                    // Re-check using Firebase's canonical email (casing/provider quirks);
reject if domain drifted.
                    String signedEmail = result.getUser() != null &&
result.getUser().getEmail() != null
                            ? result.getUser().getEmail()
                            : email;
                    if (!AuthInputValidator.isAllowedNtuLoginEmail(signedEmail)) {
                        auth.signOut();
                        UserSession.signOut(this);
                        Toast.makeText(this, R.string.auth_email_ntu_only,
Toast.LENGTH_LONG).show();
                        return;
                    }
                    if (result.getUser() != null && result.getUser().getEmail() != null) {
                        UserSession.signIn(this, result.getUser().getEmail());
                    } else {
                        UserSession.signIn(this, email);
                    }
                    LoginRememberPreferences.save(Login.this,
rememberCheck.isChecked(), rememberHint);
                    Toast.makeText(Login.this, R.string.login_success,
Toast.LENGTH_SHORT).show();
                    goToMain();
                })
                .addOnFailureListener(e -> tryLegacyFirestoreLogin(email, password,
rememberHint, e));
    }

    /**
     * Supports accounts created before Firebase Auth (password stored only in
Firestore).
     * Attempts silent migration to Firebase Auth for the next launch.
     */
    private void tryLegacyFirestoreLogin(String email, String password, String
rememberHint,
            Exception authFailure) {
```

```
        db.collection("users").document(email)
                .get()
                .addOnSuccessListener(document -> {
                    if (!document.exists()) {
                        toastAuthFailure(authFailure);
                        return;
                    }
                    String storedPassword = document.getString("password");
                    if (storedPassword != null && storedPassword.equals(password)) {
                        UserSession.signIn(Login.this, email);
                        establishFirebaseAuthAfterLegacyLogin(email, password,
rememberHint);
                        return;
                    }
                    toastAuthFailure(authFailure);
                })
                .addOnFailureListener(netErr -> {
                    if (authFailure instanceof FirebaseNetworkException
                            || netErr.getCause() instanceof FirebaseNetworkException) {
                        Toast.makeText(Login.this, R.string.auth_network_error,
Toast.LENGTH_LONG).show();
                    } else {
                        Toast.makeText(Login.this, R.string.auth_login_failed,
Toast.LENGTH_LONG).show();
                    }
                });
    }

    /** User-facing errors for failed Firebase / network cases. */
    private void toastAuthFailure(Exception e) {
        if (e instanceof FirebaseNetworkException) {
            Toast.makeText(this, R.string.auth_network_error,
Toast.LENGTH_LONG).show();
        } else if (e instanceof FirebaseAuthInvalidUserException
                || e instanceof FirebaseAuthInvalidCredentialsException) {
            Toast.makeText(this, R.string.auth_login_failed,
Toast.LENGTH_LONG).show();
        } else {
            Toast.makeText(this, R.string.auth_login_failed,
Toast.LENGTH_LONG).show();
        }
    }

    /**
     * Legacy accounts may validate via Firestore only. Firestore rules require
{@code request.auth}
```

```
     * for profile/leaderboard updates, so we finish Firebase sign-in or create before opening the app.
     */
    private void establishFirebaseAuthAfterLegacyLogin(String email, String password, String rememberHint) {
        auth.signInWithEmailAndPassword(email, password)
                .addOnSuccessListener(result -> {
                    // Same post-sign-in domain gate as loginUser (legacy row may predate this policy).
                    String signedEmail = result.getUser() != null && result.getUser().getEmail() != null
                            ? result.getUser().getEmail()
                            : email;
                    if (!AuthInputValidator.isAllowedNtuLoginEmail(signedEmail)) {
                        auth.signOut();
                        UserSession.signOut(Login.this);
                        Toast.makeText(Login.this, R.string.auth_email_ntu_only, Toast.LENGTH_LONG).show();
                        return;
                    }
                    if (result.getUser() != null && result.getUser().getEmail() != null) {
                        UserSession.signIn(Login.this, result.getUser().getEmail());
                    }
                    LoginRememberPreferences.save(Login.this, rememberCheck.isChecked(), rememberHint);
                    Toast.makeText(Login.this, R.string.login_success, Toast.LENGTH_SHORT).show();
                    goToMain();
                })
                .addOnFailureListener(signInErr -> auth.createUserWithEmailAndPassword(email, password)
                        .addOnSuccessListener(result -> {
                            // Migration path: new Auth user must still be @my.ntu.ac.uk; else remove stray account.
                            String signedEmail = result.getUser() != null && result.getUser().getEmail() != null
                                    ? result.getUser().getEmail()
                                    : email;
                            if (!AuthInputValidator.isAllowedNtuLoginEmail(signedEmail)) {
                                if (result.getUser() != null) {
                                    result.getUser().delete();
                                }
                                auth.signOut();
                                UserSession.signOut(Login.this);
                                Toast.makeText(Login.this, R.string.auth_email_ntu_only, Toast.LENGTH_LONG).show();
                                return;
```

```
                    }
                    if (result.getUser() != null && result.getUser().getEmail() !=
null) {
                        UserSession.signIn(Login.this, result.getUser().getEmail());
                    }
                    LoginRememberPreferences.save(Login.this,
rememberCheck.isChecked(), rememberHint);
                    Toast.makeText(Login.this, R.string.login_success,
Toast.LENGTH_SHORT).show();
                    goToMain();
                })
                .addOnFailureListener(createErr -> {
                    if (createErr instanceof FirebaseAuthUserCollisionException) {
                        UserSession.signOut(Login.this);
                        Log.w(TAG, "Auth account exists but password did not match
Firebase sign-in", signInErr);
                        Toast.makeText(Login.this, R.string.auth_login_failed,
Toast.LENGTH_LONG).show();
                    } else {
                        Log.e(TAG, "Could not complete Firebase Auth after Firestore
legacy login", createErr);
                        LoginRememberPreferences.save(Login.this,
rememberCheck.isChecked(), rememberHint);
                        Toast.makeText(Login.this, R.string.login_success,
Toast.LENGTH_SHORT).show();
                        goToMain();
                    }
                }));
    }

    /** Clears back stack and opens main shell ({@link navigation}). */
    private void goToMain() {
        Intent intent = new Intent(Login.this, navigation.class);
        intent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK |
Intent.FLAG_ACTIVITY_CLEAR_TASK);
        startActivity(intent);
        finish();
    }
}
```

==========
==========

FILE:
app\src\main\java\com\example\treasurehuntapplication\LoginRememberPreferences.java
==========
==========

```
package com.example.treasurehuntapplication;

import android.app.Activity;
 import android.content.Context;
 import android.content.SharedPreferences;
 import android.widget.CheckBox;
 import android.widget.EditText;

/** Persists â€œremember meâ€ (username only) for the login form. */
 public final class LoginRememberPreferences {

    private static final String PREF = "login_remember";
     private static final String KEY_REMEMBER = "remember";
     private static final String KEY_USERNAME = "username";

    private LoginRememberPreferences() {
     }

    private static SharedPreferences prefs(Context context) {
        return context.getApplicationContext().getSharedPreferences(PREF,
Context.MODE_PRIVATE);
     }

    public static void loadIntoForm(Activity activity, EditText usernameInput,
CheckBox rememberCheck) {
        SharedPreferences p = prefs(activity);
        rememberCheck.setChecked(p.getBoolean(KEY_REMEMBER, false));
        String u = p.getString(KEY_USERNAME, "");
        if (u != null && !u.isEmpty()) {
            usernameInput.setText(u);
        }
     }

    public static void save(Context context, boolean remember, String username) {
        SharedPreferences.Editor e = prefs(context).edit();
        if (remember && username != null && !username.trim().isEmpty()) {
            e.putBoolean(KEY_REMEMBER, true);
            e.putString(KEY_USERNAME, username.trim());
        } else {
            e.putBoolean(KEY_REMEMBER, false);
```

```
            e.remove(KEY_USERNAME);
        }
        e.apply();
    }

    public static void clear(Context context) {
        prefs(context).edit().clear().apply();
    }
}
```

```
================================================================
====================
FILE: app\src\main\java\com\example\treasurehuntapplication\MainActivity.java
================================================================
====================
```

```
package com.example.treasurehuntapplication;

import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

/**
 * Landing / welcome screen shown before authentication. START opens {@link
Login}.
 */
public class MainActivity extends AppCompatActivity {
    private Button Next;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);

        setContentView(R.layout.activity_main);

        // --- Primary action â†’ login ---
```

```
        Next = findViewById(R.id.Startbutton);

        Next.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent intent = new Intent(MainActivity.this, Login.class);
                startActivity(intent);
            }
        });

        // --- Edge-to-edge insets ---
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v,
windowInsets) -> {
            Insets bars =
windowInsets.getInsets(WindowInsetsCompat.Type.systemBars());
            Insets cutout =
windowInsets.getInsets(WindowInsetsCompat.Type.displayCutout());
            v.setPadding(
                    Math.max(bars.left, cutout.left),
                    Math.max(bars.top, cutout.top),
                    Math.max(bars.right, cutout.right),
                    Math.max(bars.bottom, cutout.bottom));
            return windowInsets;
        });
    }
}
```

```
================================================================
====================
FILE: app\src\main\java\com\example\treasurehuntapplication\MapFragment.java
================================================================
====================
```

```
package com.example.treasurehuntapplication;

import android.Manifest;
import android.content.pm.PackageManager;
import android.graphics.Color;
import android.location.Location;
import android.os.Bundle;
import android.os.Looper;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Toast;
```

```
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.core.app.ActivityCompat;
import androidx.core.content.ContextCompat;
import androidx.fragment.app.Fragment;

import com.google.android.gms.location.FusedLocationProviderClient;
import com.google.android.gms.location.LocationCallback;
import com.google.android.gms.location.LocationRequest;
import com.google.android.gms.location.LocationResult;
import com.google.android.gms.location.LocationServices;
import com.google.android.gms.location.Priority;
import com.google.android.gms.maps.CameraUpdateFactory;
import com.google.android.gms.maps.GoogleMap;
import com.google.android.gms.maps.OnMapReadyCallback;
import com.google.android.gms.maps.SupportMapFragment;
import com.google.android.gms.maps.model.BitmapDescriptorFactory;
import com.google.android.gms.maps.model.CircleOptions;
import com.google.android.gms.maps.model.LatLng;
import com.google.android.gms.maps.model.Marker;
import com.google.android.gms.maps.model.MarkerOptions;

public class MapFragment extends Fragment implements OnMapReadyCallback {

    /** Only the accepted hunt target pin appears, and only when the user is within
this distance. */
    private static final float PIN_VISIBLE_METERS = 40f;
    /** User must be at least this close to tap the pin and collect. */
    private static final float COLLECT_TAP_METERS = 28f;
    /** Red circle on the map shows this hunt zone (meters); visible whenever a
hunt is active. */
    private static final double SEARCH_AREA_RADIUS_METERS = 40.0;

    private static final float DEFAULT_ZOOM =
CampusMapDefaults.ZOOM_OVERVIEW;

    // --- Map & location ---
    private GoogleMap mMap;
    private FusedLocationProviderClient fusedLocationClient;
    private LocationCallback proximityCallback;
    private boolean locationUpdatesActive;
    private Location lastUserLocation;

    @Override
    public void onCreate(@Nullable Bundle savedInstanceState) {
```

```
        super.onCreate(savedInstanceState);
        // --- Fused location + callback: walk tracking + refresh markers ---
        fusedLocationClient =
LocationServices.getFusedLocationProviderClient(requireActivity());
        proximityCallback = new LocationCallback() {
            @Override
            public void onLocationResult(@NonNull LocationResult result) {
                Location loc = result.getLastLocation();
                if (loc != null) {
                    lastUserLocation = loc;

WalkDistanceTracker.recordLocation(requireContext().getApplicationContext(), loc);
                    refreshTreasureMarkers();
                }
            }
        };
    }

    @Nullable
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup
container, @Nullable Bundle savedInstanceState) {
        View view = inflater.inflate(R.layout.fragment_map, container, false);
        SupportMapFragment supportMapFragment = (SupportMapFragment)
getChildFragmentManager().findFragmentById(R.id.google_map);
        if (supportMapFragment != null) {
            supportMapFragment.getMapAsync(this);
        }
        return view;
    }

    @Override
    public void onMapReady(@NonNull GoogleMap googleMap) {
        // --- Map UI defaults & campus overview ---
        mMap = googleMap;
        mMap.getUiSettings().setZoomControlsEnabled(true);
        mMap.getUiSettings().setZoomGesturesEnabled(true);
        mMap.getUiSettings().setCompassEnabled(true);

        mMap.setOnMarkerClickListener(this::onTreasureMarkerClick);


mMap.moveCamera(CameraUpdateFactory.newLatLngZoom(CampusMapDefaults.clif
tonCampus(), DEFAULT_ZOOM));

        if (hasLocationPermission()) {
```

```
            try {
                mMap.setMyLocationEnabled(true);
                mMap.getUiSettings().setMyLocationButtonEnabled(true);
            } catch (SecurityException ignored) {
            }
            moveCameraToLastKnownOrUserOrStayOnCampus();
            seedLastLocationOnce();
        }

        refreshTreasureMarkers();
        maybeStartProximityUpdates();
    }

    /** Prime lastUserLocation from cache so markers can appear without waiting for
first update. */
    private void seedLastLocationOnce() {
        if (!hasLocationPermission()) {
            return;
        }
        try {

fusedLocationClient.getLastLocation().addOnSuccessListener(requireActivity(), loc ->
{
                if (loc != null) {
                    lastUserLocation = loc;
                    refreshTreasureMarkers();
                }
            });
        } catch (SecurityException ignored) {
        }
    }

    @Override
    public void onResume() {
        super.onResume();
        if (mMap != null) {
            refreshTreasureMarkers();
            maybeStartProximityUpdates();
        }
    }

    @Override
    public void onPause() {
        super.onPause();
        stopProximityUpdates();
    }
```

```
    /** Fine location granted â€” required for my-location layer and hunt logic. */
    private boolean hasLocationPermission() {
        return ActivityCompat.checkSelfPermission(requireContext(),
Manifest.permission.ACCESS_FINE_LOCATION) ==
PackageManager.PERMISSION_GRANTED;
    }

    /** Prefer device position when near campus; otherwise stay on Clifton default.
*/
    private void moveCameraToLastKnownOrUserOrStayOnCampus() {
        if (!hasLocationPermission()) {
            return;
        }
        try {

fusedLocationClient.getLastLocation().addOnSuccessListener(requireActivity(),
location -> {
                if (mMap == null) {
                    return;
                }
                if (location != null &&
CampusMapDefaults.shouldUseDeviceLocationForCamera(location)) {
                    lastUserLocation = location;
                    LatLng cur = new LatLng(location.getLatitude(),
location.getLongitude());
                    mMap.animateCamera(CameraUpdateFactory.newLatLngZoom(cur,
DEFAULT_ZOOM));
                } else if (location == null) {
                    requestFreshLocationOrCampus();
                } else {
                    lastUserLocation = location;
                }
            });
        } catch (SecurityException ignored) {
        }
    }

    /** If last location is null, try a fresh high-accuracy read before falling back to
campus. */
    private void requestFreshLocationOrCampus() {
        if (!hasLocationPermission()) {
            return;
        }
        try {
```

```
fusedLocationClient.getCurrentLocation(Priority.PRIORITY_HIGH_ACCURACY, null)
                .addOnSuccessListener(location -> {
                    if (mMap == null) {
                        return;
                    }
                    if (location != null &&
CampusMapDefaults.shouldUseDeviceLocationForCamera(location)) {
                        lastUserLocation = location;
                        LatLng cur = new LatLng(location.getLatitude(),
location.getLongitude());

mMap.animateCamera(CameraUpdateFactory.newLatLngZoom(cur,
DEFAULT_ZOOM));
                    } else {
                        if (location != null) {
                            lastUserLocation = location;
                        }

mMap.moveCamera(CameraUpdateFactory.newLatLngZoom(CampusMapDefaults.clif
tonCampus(), DEFAULT_ZOOM));
                    }
                    refreshTreasureMarkers();
                });
        } catch (SecurityException ignored) {
        }
    }

    /**
     * Draws the accepted active treasure: a red search circle at the target always (so
you can pan to it),
     * plus a tappable pin when you are within {@link #PIN_VISIBLE_METERS}.
     */
    private void refreshTreasureMarkers() {
        if (mMap == null) {
            return;
        }
        mMap.clear();
        if (hasLocationPermission()) {
            try {
                mMap.setMyLocationEnabled(true);
                mMap.getUiSettings().setMyLocationButtonEnabled(true);
            } catch (SecurityException ignored) {
            }
        }
```

```
        Treasure active =
ActiveTreasureManager.getActiveTreasure(requireContext());
        if (active == null || TreasureManager.isFound(requireContext(),
active.getId())) {
            return;
        }

        LatLng pos = new LatLng(active.getLatitude(), active.getLongitude());

        int strokeRgb = ContextCompat.getColor(requireContext(), R.color.ntuRed);
        int fill = Color.argb(55, Color.red(strokeRgb), Color.green(strokeRgb),
Color.blue(strokeRgb));
        mMap.addCircle(new CircleOptions()
                .center(pos)
                .radius(SEARCH_AREA_RADIUS_METERS)
                .strokeColor(strokeRgb)
                .strokeWidth(5f)
                .fillColor(fill));

        if (lastUserLocation == null) {
            return;
        }

        float[] distance = new float[1];
        Location.distanceBetween(
                lastUserLocation.getLatitude(),
                lastUserLocation.getLongitude(),
                active.getLatitude(),
                active.getLongitude(),
                distance);
        if (distance[0] > PIN_VISIBLE_METERS) {
            return;
        }

        MarkerOptions mo = new MarkerOptions()
                .position(pos)
                .title(active.getTitle())
                .snippet(getString(R.string.map_pin_snippet))
                .icon(BitmapDescriptorFactory.defaultMarker(BitmapDescriptorFactory.HU
E_AZURE));
        Marker marker = mMap.addMarker(mo);
        if (marker != null) {
            marker.setTag(active.getId());
        }
    }
```

```
    /** Distance-gated collect dialog; ignores wrong marker / inactive hunts. */
    private boolean onTreasureMarkerClick(@NonNull Marker marker) {
        Object tag = marker.getTag();
        if (!(tag instanceof Integer)) {
            return false;
        }
        int treasureId = (Integer) tag;
        Treasure treasure = TreasureManager.getById(treasureId);
        if (treasure == null || TreasureManager.isFound(requireContext(), treasureId))
{
            return true;
        }
        if (!ActiveTreasureManager.isActive(requireContext(), treasureId)) {
            return true;
        }
        if (lastUserLocation == null) {
            Toast.makeText(requireContext(), R.string.map_wait_for_location,
Toast.LENGTH_SHORT).show();
            return true;
        }
        float[] d = new float[1];
        Location.distanceBetween(
                lastUserLocation.getLatitude(),
                lastUserLocation.getLongitude(),
                treasure.getLatitude(),
                treasure.getLongitude(),
                d);
        if (d[0] > COLLECT_TAP_METERS) {
            Toast.makeText(requireContext(), R.string.map_too_far_to_collect,
Toast.LENGTH_SHORT).show();
            return true;
        }

        NtuMaterialDialogs.builder(requireContext())
                .setTitle(R.string.map_collect_title)
                .setMessage(getString(R.string.map_collect_message,
treasure.getTitle(), treasure.getPoints()))
                .setPositiveButton(R.string.map_collect_confirm, (dialog, which) ->
onTreasureCollected(treasure))
                .setNegativeButton(android.R.string.cancel, null)
                .show();
        return true;
    }

    /** Periodic GPS updates while map is visible (walk distance + marker refresh).
*/
```

```
    private void maybeStartProximityUpdates() {
        if (!hasLocationPermission() || mMap == null) {
            return;
        }
        Treasure active =
ActiveTreasureManager.getActiveTreasure(requireContext());
        if (active == null || TreasureManager.isFound(requireContext(),
active.getId())) {
            stopProximityUpdates();
            return;
        }
        if (locationUpdatesActive) {
            return;
        }
        LocationRequest request = new
LocationRequest.Builder(Priority.PRIORITY_HIGH_ACCURACY, 3_000L)
                .setMinUpdateIntervalMillis(2_000L)
                .setMaxUpdateDelayMillis(8_000L)
                .build();
        try {
            fusedLocationClient.requestLocationUpdates(request, proximityCallback,
Looper.getMainLooper());
            locationUpdatesActive = true;
        } catch (SecurityException e) {
            locationUpdatesActive = false;
        }
    }

    /** Stops GPS updates when fragment pauses to save battery. */
    private void stopProximityUpdates() {
        if (!locationUpdatesActive) {
            return;
        }
        try {
            fusedLocationClient.removeLocationUpdates(proximityCallback);
        } catch (SecurityException ignored) {
        }
        locationUpdatesActive = false;
    }

    /** Persists found state, points, clears active hunt, syncs Firestore, refreshes
map. */
    private void onTreasureCollected(@NonNull Treasure treasure) {
        TreasureManager.markFound(requireContext(), treasure.getId());
        ActiveTreasureManager.clearActiveTreasure(requireContext());
        ScoreRepository.addPoints(requireContext(), treasure.getPoints());
```

```
        UserGameStateFirestoreSync.pushGameStateToFirestore(requireContext());
        Toast.makeText(requireContext(),
                getString(R.string.toast_treasure_found, treasure.getPoints()),
                Toast.LENGTH_LONG).show();
        stopProximityUpdates();
        refreshTreasureMarkers();
    }
}
```

```
================================================================================
FILE: app\src\main\java\com\example\treasurehuntapplication\navigation.java
================================================================================

package com.example.treasurehuntapplication;

import android.Manifest;
import android.content.Intent;
import android.content.SharedPreferences;
import android.content.pm.PackageManager;
import android.content.res.ColorStateList;
import android.os.Bundle;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;
import androidx.core.content.ContextCompat;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

import com.example.treasurehuntapplication.databinding.ActivityNavigationBinding;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.android.material.navigation.NavigationBarView;

/**
 * Main app shell after login: bottom navigation, fragment swapping (Map, Treasure,
Leaderboard,
 * Profile, Support), session sync with Firebase, one-time game-data migration,
location permission,
 * and first-run tutorial per user.
 */
```

```
public class navigation extends AppCompatActivity {

    private static final int REQ_LOCATION = 100;

    // --- One-time local + remote game reset (versioned) ---
    /**
     * Bump when shipping a full reset of local + (for Firebase users) remote game
fields for all installs.
     */
    private static final int GAME_DATA_VERSION = 2;
    private static final String BOOTSTRAP_PREF = "app_bootstrap";
    private static final String KEY_GAME_DATA_VERSION = "game_data_version";

    ActivityNavigationBinding binding;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        // --- Align UserSession with Firebase if app was restored with persisted Auth -
--
        syncSessionWithFirebaseAuth();
        // --- Optional shipped reset: clear prefs + wipe current userâ€™s Firestore
game fields once ---
        applyGameDataMigrationIfNeeded();
        // --- Pull display name from Firestore when account changes ---
        UserProfileFirestoreSync.hydrateDisplayNameForCurrentAccount(this);
        // --- Not signed in â†’ back to Login ---
        if (!UserSession.isSignedIn(this)) {
            Intent login = new Intent(this, Login.class);
            login.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK |
Intent.FLAG_ACTIVITY_CLEAR_TASK);
            startActivity(login);
            finish();
            return;
        }

        // --- Ensure users/{email} exists for leaderboard visibility ---
        LeaderboardUserRegistration.ensureRegistered(this);

        EdgeToEdge.enable(this);

        // --- Map / hunt needs location ---
        requestLocationIfNeeded();
```

```
        binding = ActivityNavigationBinding.inflate(getLayoutInflater());
        setContentView(binding.getRoot());

        NavigationBarView navBar = binding.Navbar;
        navBar.setItemActiveIndicatorEnabled(true);
        navBar.setItemActiveIndicatorColor(
                ColorStateList.valueOf(ContextCompat.getColor(this,
R.color.ntu_nav_indicator)));

        // --- First launch per account: tutorial after Map is laid out (dimmed map
behind) ---
        binding.getRoot().post(() -> TutorialPopup.showIfFirstTime(this));

        // --- Edge-to-edge insets on main content ---
        ViewCompat.setOnApplyWindowInsetsListener(binding.getRoot(), (v,
windowInsets) -> {
            Insets bars =
windowInsets.getInsets(WindowInsetsCompat.Type.systemBars());
            Insets cutout =
windowInsets.getInsets(WindowInsetsCompat.Type.displayCutout());
            v.setPadding(
                    Math.max(bars.left, cutout.left),
                    Math.max(bars.top, cutout.top),
                    Math.max(bars.right, cutout.right),
                    Math.max(bars.bottom, cutout.bottom));
            return windowInsets;
        });

        // --- Initial tab: Map ---
        if (savedInstanceState == null) {
            getSupportFragmentManager().beginTransaction()
                    .replace(R.id.fragment_container, new MapFragment())
                    .commit();
            binding.Navbar.setSelectedItemId(R.id.map);
        }

        // --- Bottom nav: swap fragments ---
        binding.Navbar.setOnItemSelectedListener(item -> {
            int itemId = item.getItemId();
            if (itemId == R.id.profile) {
                getSupportFragmentManager().beginTransaction()
                        .replace(R.id.fragment_container, new ProfileFragment())
                        .commit();
            } else if (itemId == R.id.leaderboard) {
                getSupportFragmentManager().beginTransaction()
                        .replace(R.id.fragment_container, new LeaderboardFragment())
```

```
                .commit();
        } else if (itemId == R.id.map) {
            getSupportFragmentManager().beginTransaction()
                .replace(R.id.fragment_container, new MapFragment())
                .commit();
        } else if (itemId == R.id.treasure) {
            getSupportFragmentManager().beginTransaction()
                .replace(R.id.fragment_container, new TreasureFragment())
                .commit();
        } else if (itemId == R.id.support) {
            getSupportFragmentManager().beginTransaction()
                .replace(R.id.fragment_container, new SupportFragment())
                .commit();
        }
        return true;
    });
}

/** One-time wipe of local progress and (for Firebase users) remote score / walk
/ treasures. */
private void applyGameDataMigrationIfNeeded() {
    SharedPreferences p = getSharedPreferences(BOOTSTRAP_PREF,
MODE_PRIVATE);
    int stored = p.getInt(KEY_GAME_DATA_VERSION, 0);
    if (stored >= GAME_DATA_VERSION) {
        return;
    }
    GameStateReset.clearLocalGameProgress(this);
    UserGameStateFirestoreSync.wipeRemoteGameFieldsForCurrentUser(this);
    p.edit().putInt(KEY_GAME_DATA_VERSION, GAME_DATA_VERSION).apply();
}

/** Keeps local session aligned when the user signed in with Firebase Auth. */
private void syncSessionWithFirebaseAuth() {
    FirebaseUser u = FirebaseAuth.getInstance().getCurrentUser();
    if (u != null && u.getEmail() != null && !u.getEmail().isEmpty()) {
        UserSession.signIn(this, u.getEmail());
    }
}

@Override
protected void onResume() {
    super.onResume();
    // --- Session + profile may change while app is backgrounded ---
    syncSessionWithFirebaseAuth();
    UserProfileFirestoreSync.hydrateDisplayNameForCurrentAccount(this);
```

```
        if (UserSession.isSignedIn(this)) {
            LeaderboardUserRegistration.ensureRegistered(this);
            UserGameStateFirestoreSync.pushGameStateToFirestore(this);
        }
    }

    /** Requests fine/coarse location if not already granted (treasure hunt map). */
    private void requestLocationIfNeeded() {
        if (ContextCompat.checkSelfPermission(this,
Manifest.permission.ACCESS_FINE_LOCATION)
                != PackageManager.PERMISSION_GRANTED) {
            ActivityCompat.requestPermissions(this,
                    new String[]{
                            Manifest.permission.ACCESS_FINE_LOCATION,
                            Manifest.permission.ACCESS_COARSE_LOCATION
                    },
                    REQ_LOCATION);
        }
    }

    /** After accepting a treasure, jump to Map so GPS hunt continues there. */
    public void showMapTab() {
        if (binding.Navbar.getSelectedItemId() == R.id.map) {
            getSupportFragmentManager().beginTransaction()
                    .replace(R.id.fragment_container, new MapFragment())
                    .commit();
        } else {
            binding.Navbar.setSelectedItemId(R.id.map);
        }
    }
}
```

```
==================================================
====================
FILE:
app\src\main\java\com\example\treasurehuntapplication\NtuMaterialDialogs.java
==================================================
====================

package com.example.treasurehuntapplication;

import android.content.Context;
 import android.graphics.drawable.ColorDrawable;
```

```
import androidx.annotation.NonNull;
 import androidx.appcompat.app.AlertDialog;

import com.google.android.material.dialog.MaterialAlertDialogBuilder;

/** Shared NTU-styled {@link MaterialAlertDialogBuilder} and window polish for
custom views. */
 public final class NtuMaterialDialogs {

    private NtuMaterialDialogs() {
     }

    @NonNull
     public static MaterialAlertDialogBuilder builder(@NonNull Context context) {
         return new MaterialAlertDialogBuilder(context,
R.style.ThemeOverlay_NTU_MaterialAlertDialog);
     }

    /**
      * Use with {@link MaterialAlertDialogBuilder#setView} so the red-bordered card
(or hint layout)
      * isnâ€™t clipped by the default dialog panel.
      */
     public static void applyCustomContentWindow(@NonNull AlertDialog dialog, float
dimAmount) {
         if (dialog.getWindow() != null) {
             dialog.getWindow().setBackgroundDrawable(new
ColorDrawable(android.graphics.Color.TRANSPARENT));
             dialog.getWindow().setDimAmount(dimAmount);
         }
     }
 }
```

```
================================================================================
FILE: app\src\main\java\com\example\treasurehuntapplication\ProfileFragment.java
================================================================================

package com.example.treasurehuntapplication;

import android.content.Context;
import android.content.Intent;
```

```
import android.content.res.ColorStateList;
import android.graphics.Bitmap;
import android.graphics.BitmapFactory;
import android.graphics.Color;
import android.graphics.Typeface;
import android.os.Bundle;
import android.util.LruCache;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.view.Gravity;
import android.widget.FrameLayout;
import android.widget.GridLayout;
import android.widget.ImageView;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.appcompat.content.res.AppCompatResources;
import androidx.core.content.ContextCompat;
import androidx.core.widget.ImageViewCompat;
import androidx.fragment.app.Fragment;

import com.google.android.material.card.MaterialCardView;
import com.google.firebase.auth.FirebaseAuth;

import com.example.treasurehuntapplication.Treasure.Rarity;

public class ProfileFragment extends Fragment {

    private static final int BADGE_TARGET_PX = 200;
    private static final LruCache<Integer, Bitmap> BADGE_BITMAP_CACHE = new
LruCache<>(
            Math.max(8, (int) (Runtime.getRuntime().maxMemory() / 1024L / 12L))
    );

    private TextView commonCount;
    private TextView rareCount;
    private TextView legendaryCount;
    private TextView totalPointsText;
    private TextView profileName;
    private TextView profileLocation;
    private TextView profileCourse;
    private TextView profilePronouns;
    private ImageView profileImage;
    private GridLayout badgeGrid;
    private GridLayout treasureGrid;
```

```
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, ViewGroup container,
                             Bundle savedInstanceState) {

        View view = inflater.inflate(R.layout.fragment_profile, container, false);

        // --- Grids & text views ---
        badgeGrid = view.findViewById(R.id.badgeGrid);
        treasureGrid = view.findViewById(R.id.treasureGrid);
        commonCount = view.findViewById(R.id.commonCount);
        rareCount = view.findViewById(R.id.rareCount);
        legendaryCount = view.findViewById(R.id.legendaryCount);
        totalPointsText = view.findViewById(R.id.totalPointsText);
        profileName = view.findViewById(R.id.profileName);
        profileLocation = view.findViewById(R.id.profileLocation);
        profileCourse = view.findViewById(R.id.profileCourse);
        profilePronouns = view.findViewById(R.id.profilePronouns);
        profileImage = view.findViewById(R.id.profileImage);

        // --- Navigation: settings fragment ---
        view.findViewById(R.id.btnOpenSettings).setOnClickListener(v ->
                requireActivity().getSupportFragmentManager().beginTransaction()
                        .replace(R.id.fragment_container, new SettingsFragment())
                        .addToBackStack(null)
                        .commit());

        // --- Sign out â†’ Login ---
        view.findViewById(R.id.btnSignOut).setOnClickListener(v ->
confirmSignOut());

        refreshStats(view);
        return view;
    }

    @Override
    public void onResume() {
        super.onResume();
        UserGameStateFirestoreSync.pushGameStateToFirestore(requireContext());
        if (getView() != null) {
            refreshStats(getView());
        }
    }

    /** Confirms before clearing Firebase + local session state. */
    private void confirmSignOut() {
```

```
        NtuMaterialDialogs.builder(requireContext())
                .setTitle(R.string.logout_title)
                .setMessage(R.string.logout_message)
                .setPositiveButton(R.string.logout_confirm, (d, w) -> performSignOut())
                .setNegativeButton(R.string.settings_cancel, null)
                .show();
    }

    /** Full logout: Auth, prefs, remember-me, return to {@link Login}. */
    private void performSignOut() {
        FirebaseAuth.getInstance().signOut();
        ActiveTreasureManager.clearActiveTreasure(requireContext());
        UserProfileFirestoreSync.clearAccountHydrationMarker(requireContext());
        UserSession.signOut(requireContext());
        LoginRememberPreferences.clear(requireContext());
        Intent i = new Intent(requireActivity(), Login.class);
        i.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK |
Intent.FLAG_ACTIVITY_CLEAR_TASK);
        startActivity(i);
        requireActivity().finish();
    }

    /** Binds profile fields, rarity counts, points, badges, and treasure mini-cells. */
    private void refreshStats(View view) {
        String email = UserSession.getEmail(requireContext());
        String displayName = ProfilePreferences.getDisplayName(requireContext());
        if (displayName != null && !displayName.isEmpty()) {
            profileName.setText(displayName);
        } else {
            profileName.setText(email != null ? email :
getString(R.string.stats_guest_name));
        }

        String loc = ProfilePreferences.getLocation(requireContext());
        profileLocation.setText(loc == null || loc.isEmpty()
                ? getString(R.string.stats_profile_campus_line)
                : loc);

        String course = ProfilePreferences.getCourse(requireContext());
        if (course != null && !course.isEmpty()) {
            profileCourse.setVisibility(View.VISIBLE);
            profileCourse.setText(course);
        } else {
            profileCourse.setVisibility(View.GONE);
        }
```

```
        String pronouns = ProfilePreferences.getPronouns(requireContext());
        if (pronouns != null && !pronouns.isEmpty()) {
            profilePronouns.setVisibility(View.VISIBLE);
            profilePronouns.setText(pronouns);
        } else {
            profilePronouns.setVisibility(View.GONE);
        }

        profileImage.setImageResource(R.drawable.baseline_person_24);
        ImageViewCompat.setImageTintList(profileImage,
                AppCompatResources.getColorStateList(requireContext(),
R.color.ntu_support_text_muted));

        int foundCommon = countFound(requireContext(), Rarity.COMMON);
        int foundRare = countFound(requireContext(), Rarity.RARE);
        int foundLeg = countFound(requireContext(), Rarity.LEGENDARY);
        int totalCommon = TreasureManager.countByRarity(Rarity.COMMON);
        int totalRare = TreasureManager.countByRarity(Rarity.RARE);
        int totalLeg = TreasureManager.countByRarity(Rarity.LEGENDARY);

        commonCount.setText(getString(R.string.stats_rarity_count, foundCommon,
totalCommon));
        rareCount.setText(getString(R.string.stats_rarity_count, foundRare,
totalRare));
        legendaryCount.setText(getString(R.string.stats_rarity_count, foundLeg,
totalLeg));

        ScoreRepository.getDisplayScore(requireContext(), score ->
                totalPointsText.setText(getString(R.string.stats_total_points,
String.valueOf(score))));

        badgeGrid.removeAllViews();
        long walked = WalkDistanceTracker.getTotalMeters(requireContext());
        int foundTotal = TreasureManager.getFoundCount(requireContext());
        for (WalkAchievements.Badge b : WalkAchievements.getWalkBadges()) {
            addWalkAchievementBadge(badgeGrid, b, walked);
        }
        for (TreasureAchievements.Badge b :
TreasureAchievements.getTreasureBadges()) {
            addTreasureAchievementBadge(badgeGrid, b, foundTotal);
        }

        treasureGrid.removeAllViews();
        for (Treasure t : TreasureManager.getAll()) {
            addTreasureCell(treasureGrid, t);
        }
```

```
    }

    /** How many treasures of this rarity are marked found for current user. */
    private static int countFound(Context context, Rarity rarity) {
        int c = 0;
        for (Treasure t : TreasureManager.getByRarity(rarity)) {
            if (TreasureManager.isFound(context, t.getId())) {
                c++;
            }
        }
        return c;
    }

    private void addWalkAchievementBadge(GridLayout grid,
WalkAchievements.Badge b, long walkedMeters) {
        boolean unlocked = walkedMeters >= b.requiredMeters;
        HexagonImageView badge = createHexBadgeView(grid, b.drawableRes,
unlocked, getString(b.labelRes));
        badge.setOnClickListener(v -> showAchievementDetail(
                b.labelRes, b.detailRes, unlocked, true, walkedMeters, 0,
b.requiredMeters));
        grid.addView(badge);
    }

    private void addTreasureAchievementBadge(GridLayout grid,
TreasureAchievements.Badge b, int foundTotal) {
        boolean unlocked = foundTotal >= b.requiredFound;
        HexagonImageView badge = createHexBadgeView(grid, b.drawableRes,
unlocked, getString(b.labelRes));
        badge.setOnClickListener(v -> showAchievementDetail(
                b.labelRes, b.detailRes, unlocked, false, 0, foundTotal,
b.requiredFound));
        grid.addView(badge);
    }

    private HexagonImageView createHexBadgeView(GridLayout grid, int
drawableRes, boolean unlocked,
            String contentDescription) {
        HexagonImageView badge = new HexagonImageView(requireContext());
        GridLayout.LayoutParams params = new GridLayout.LayoutParams();
        params.width = dp(75);
        params.height = dp(75);
        params.setMargins(dp(3), dp(3), dp(3), dp(3));
        badge.setLayoutParams(params);
        setBadgeImage(badge, drawableRes);
        badge.setContentDescription(contentDescription);
```

```
        badge.setStrokeWidthDp(2f);
        badge.setStrokeColor(ContextCompat.getColor(requireContext(),
                unlocked ? R.color.ntuRed : R.color.ntu_support_text_muted));
        badge.setAlpha(unlocked ? 1f : 0.25f);
        if (unlocked) {
            badge.clearColorFilter();
        } else {
            badge.setColorFilter(Color.GRAY);
        }
        badge.setClickable(true);
        badge.setFocusable(true);
        return badge;
    }

    private void showAchievementDetail(int titleRes, int detailRes, boolean unlocked,
boolean isWalkBadge,
            long walkedMeters, int treasuresFound, int requirement) {
        String body = getString(detailRes);
        if (unlocked) {
            body += "\n\n" + getString(R.string.achievement_status_unlocked);
        } else if (isWalkBadge) {
            int walkedShown = walkedMeters > Integer.MAX_VALUE ?
Integer.MAX_VALUE : (int) walkedMeters;
            body += "\n\n" + getString(R.string.achievement_progress_walk,
walkedShown, requirement);
        } else {
            body += "\n\n" + getString(R.string.achievement_progress_treasure,
treasuresFound, requirement);
        }
        NtuMaterialDialogs.builder(requireContext())
                .setTitle(titleRes)
                .setMessage(body)
                .setPositiveButton(R.string.achievement_dialog_ok, null)
                .show();
    }

    /** Downsamples badge art via inSampleSize and caches bitmaps in {@link
#BADGE_BITMAP_CACHE}. */
    private void setBadgeImage(@NonNull ImageView target, int drawableRes) {
        Bitmap cached = BADGE_BITMAP_CACHE.get(drawableRes);
        if (cached != null && !cached.isRecycled()) {
            target.setImageBitmap(cached);
            return;
        }

        BitmapFactory.Options bounds = new BitmapFactory.Options();
```

```
        bounds.inJustDecodeBounds = true;
        BitmapFactory.decodeResource(getResources(), drawableRes, bounds);

        BitmapFactory.Options opts = new BitmapFactory.Options();
        opts.inPreferredConfig = Bitmap.Config.RGB_565;
        opts.inDither = true;
        opts.inSampleSize = computeInSampleSize(bounds, BADGE_TARGET_PX,
BADGE_TARGET_PX);

        Bitmap bmp = BitmapFactory.decodeResource(getResources(), drawableRes,
opts);
        if (bmp != null) {
            BADGE_BITMAP_CACHE.put(drawableRes, bmp);
            target.setImageBitmap(bmp);
        } else {
            target.setImageResource(drawableRes);
        }
    }

    private static int computeInSampleSize(@NonNull BitmapFactory.Options bounds,
int reqWidth, int reqHeight) {
        int height = Math.max(1, bounds.outHeight);
        int width = Math.max(1, bounds.outWidth);
        int inSampleSize = 1;

        if (height > reqHeight || width > reqWidth) {
            int halfHeight = height / 2;
            int halfWidth = width / 2;

            while ((halfHeight / inSampleSize) >= reqHeight && (halfWidth /
inSampleSize) >= reqWidth) {
                inSampleSize *= 2;
            }
        }
        return Math.max(1, inSampleSize);
    }

    /** Small square in â€œall treasuresâ€ grid; shows collected overlay when found.
*/
    private void addTreasureCell(GridLayout grid, Treasure treasure) {
        MaterialCardView wrapper = new MaterialCardView(requireContext());
        GridLayout.LayoutParams wrapParams = new GridLayout.LayoutParams();
        wrapParams.width = dp(65);
        wrapParams.height = dp(65);
        wrapParams.setMargins(dp(6), dp(6), dp(6), dp(6));
        wrapper.setLayoutParams(wrapParams);
```

```
        wrapper.setRadius(dp(12));
        wrapper.setCardElevation(0f);
        wrapper.setStrokeWidth(dp(1));
        wrapper.setStrokeColor(ColorStateList.valueOf(
                ContextCompat.getColor(requireContext(),
R.color.ntu_leaderboard_row_stroke)));
        wrapper.setCardBackgroundColor(tileColor(treasure.getRarity()));
        wrapper.setPreventCornerOverlap(false);
        wrapper.setUseCompatPadding(false);

        ImageView slot = new ImageView(requireContext());
        FrameLayout.LayoutParams imgParams = new FrameLayout.LayoutParams(
                FrameLayout.LayoutParams.MATCH_PARENT,
                FrameLayout.LayoutParams.MATCH_PARENT);
        slot.setLayoutParams(imgParams);
        slot.setScaleType(ImageView.ScaleType.CENTER_CROP);
        slot.setImageResource(R.drawable.greyplaceholder);
        boolean found = TreasureManager.isFound(requireContext(),
treasure.getId());
        slot.setAlpha(found ? 1f : 0.4f);
        wrapper.addView(slot);

        if (found) {
            TextView badge = new TextView(requireContext());
            badge.setText(R.string.treasure_collected_badge);
            badge.setTextColor(Color.WHITE);
            badge.setTextSize(9f);
            badge.setTypeface(null, Typeface.BOLD);
            badge.setGravity(Gravity.CENTER);
            badge.setBackgroundColor(0xDD000000);
            FrameLayout.LayoutParams badgeParams = new
FrameLayout.LayoutParams(
                    FrameLayout.LayoutParams.MATCH_PARENT,
                    FrameLayout.LayoutParams.WRAP_CONTENT);
            badgeParams.gravity = Gravity.BOTTOM;
            badge.setPadding(2, 4, 2, 4);
            wrapper.addView(badge, badgeParams);
        }

        grid.addView(wrapper);
    }

    /** Card background by rarity (aligned with {@link TreasureFragment}). */
    private static int tileColor(Rarity rarity) {
        switch (rarity) {
            case COMMON:
```

```
            return Color.rgb(76, 175, 80);
        case RARE:
            return Color.rgb(33, 150, 243);
        case LEGENDARY:
            return Color.rgb(255, 193, 7);
        default:
            return Color.DKGRAY;
      }
   }

   /** Density-independent pixels â†’ px for programmatic layouts. */
   private int dp(int d) {
      float density = getResources().getDisplayMetrics().density;
      return Math.round(d * density);
   }
}
```

```
==========
==========
FILE:
app\src\main\java\com\example\treasurehuntapplication\ProfilePreferences.java
==========
==========

package com.example.treasurehuntapplication;

import android.content.Context;
 import android.content.SharedPreferences;

/** Local profile text fields, scoped by {@link UserSession} email. */
 public final class ProfilePreferences {

    private static final String PREF = "profile_fields";
     private static final String KEY_DISPLAY = "display_";
     private static final String KEY_LOCATION = "location_";
     private static final String KEY_COURSE = "course_";
     private static final String KEY_PRONOUNS = "pronouns_";

    private ProfilePreferences() {
     }

    private static SharedPreferences prefs(Context context) {
        return context.getApplicationContext().getSharedPreferences(PREF,
```

```
Context.MODE_PRIVATE);
    }

    private static String safeKey(String raw) {
        return raw.trim().toLowerCase().replaceAll("[^a-z0-9]+", "_");
    }

    private static String suffix(Context context) {
        String email = UserSession.getEmail(context.getApplicationContext());
        if (email == null || email.isEmpty()) {
            return "guest";
        }
        return safeKey(email);
    }

    public static String getDisplayName(Context context) {
        return prefs(context).getString(KEY_DISPLAY + suffix(context), "");
    }

    public static void setDisplayName(Context context, String value) {
        prefs(context).edit().putString(KEY_DISPLAY + suffix(context), value != null ?
value : "").apply();
    }

    public static String getLocation(Context context) {
        return prefs(context).getString(KEY_LOCATION + suffix(context), "");
    }

    public static void setLocation(Context context, String value) {
        prefs(context).edit().putString(KEY_LOCATION + suffix(context), value !=
null ? value : "").apply();
    }

    public static String getCourse(Context context) {
        return prefs(context).getString(KEY_COURSE + suffix(context), "");
    }

    public static void setCourse(Context context, String value) {
        prefs(context).edit().putString(KEY_COURSE + suffix(context), value != null ?
value : "").apply();
    }

    public static String getPronouns(Context context) {
        return prefs(context).getString(KEY_PRONOUNS + suffix(context), "");
    }
```

```
    public static void setPronouns(Context context, String value) {
        prefs(context).edit().putString(KEY_PRONOUNS + suffix(context), value !=
null ? value : "").apply();
    }
 }
```

```
================================================================
====================
FILE:
app\src\main\java\com\example\treasurehuntapplication\PublicPlayerStatsDialog.ja
va
================================================================
====================
```

```
package com.example.treasurehuntapplication;

import android.view.LayoutInflater;
 import android.view.View;
 import android.widget.TextView;
 import android.widget.Toast;

import androidx.appcompat.app.AlertDialog;
 import androidx.fragment.app.Fragment;

import com.google.firebase.firestore.DocumentSnapshot;
 import com.google.firebase.firestore.FirebaseFirestore;

import java.util.List;
 import java.util.Locale;

import de.hdodenhof.circleimageview.CircleImageView;

/** Loads another playerâ€™s public Firestore profile for the leaderboard row tap.
*/
 public final class PublicPlayerStatsDialog {

    private PublicPlayerStatsDialog() {
     }

    public static void show(Fragment parent, LeaderboardModel user) {
        if (!parent.isAdded()) {
            return;
```

```
        }
        String docId = user.getFirestoreDocumentId();
        if (docId == null || docId.isEmpty()) {
            Toast.makeText(parent.requireContext(),
R.string.public_player_profile_unavailable,
                    Toast.LENGTH_SHORT).show();
            return;
        }

       AlertDialog loading = NtuMaterialDialogs.builder(parent.requireContext())
                .setTitle(R.string.public_player_dialog_title)
                .setMessage(R.string.public_player_loading)
                .setCancelable(true)
                .create();
        loading.show();

       FirebaseFirestore.getInstance()
                .collection("users")
                .document(docId)
                .get()
                .addOnCompleteListener(task -> {
                    if (!parent.isAdded()) {
                        return;
                    }
                    loading.dismiss();
                    if (!task.isSuccessful() || task.getResult() == null
|| !task.getResult().exists()) {
                        Toast.makeText(parent.requireContext(),
R.string.public_player_load_failed,
                                Toast.LENGTH_LONG).show();
                        return;
                    }
                    DocumentSnapshot doc = task.getResult();
                    String name = doc.getString("username");
                    if (name == null || name.isEmpty()) {
                        name = user.getDisplayName();
                    }
                    Long scoreObj = doc.getLong("score");
                    long score = scoreObj != null ? scoreObj : 0L;
                    int nFound = parseFoundCount(doc);

                   String achievements =
parent.getString(R.string.public_player_achievements_heading)
                            + "\nÂ· " + parent.getString(R.string.achievement_treasure_1)
                            + "\nÂ· " +
parent.getString(R.string.achievement_walk_100m);
```

```
                String treasuresHeading =
parent.getString(R.string.public_player_treasures_heading);
                String body = String.format(Locale.getDefault(),
                        "%s\n\n%s: %d\n\n%s\n\n%s: %d",
                        name,
                        parent.getString(R.string.leaderboard_title),
                        score,
                        achievements,
                        treasuresHeading,
                        nFound);

                View content = LayoutInflater.from(parent.requireContext())
                        .inflate(R.layout.dialog_public_player, null);
                CircleImageView avatar =
content.findViewById(R.id.publicPlayerAvatar);
                ((TextView)
content.findViewById(R.id.publicPlayerBody)).setText(body);
                avatar.setImageResource(R.drawable.baseline_person_24);
                AlertDialog dialog =
NtuMaterialDialogs.builder(parent.requireContext())
                        .setView(content)
                        .create();
                NtuMaterialDialogs.applyCustomContentWindow(dialog, 0.65f);
                content.findViewById(R.id.publicPlayerOk).setOnClickListener(v ->
dialog.dismiss());
                dialog.show();
            });
    }

   private static int parseFoundCount(DocumentSnapshot doc) {
        Object raw = doc.get("foundTreasureIds");
        if (!(raw instanceof List<?>)) {
            return 0;
        }
        List<?> list = (List<?>) raw;
        return list.size();
    }
 }
```

```
==================================================
====================
FILE:
app\src\main\java\com\example\treasurehuntapplication\ScoreRepository.java
```

```
================================================================
====================

package com.example.treasurehuntapplication;

import android.content.Context;
 import android.content.SharedPreferences;
 import android.os.Handler;
 import android.os.Looper;

import java.util.function.Consumer;

/** Per-user score stored locally; Firestore sync is handled by {@link
UserGameStateFirestoreSync}. */
 public final class ScoreRepository {

    private static final String PREF = "score_repository";
     private static final String KEY_SCORE = "score_";

    private ScoreRepository() {
     }

    private static SharedPreferences prefs(Context context) {
        return context.getApplicationContext().getSharedPreferences(PREF,
Context.MODE_PRIVATE);
     }

    private static String safeKey(String raw) {
        return raw.trim().toLowerCase().replaceAll("[^a-z0-9]+", "_");
     }

    private static String keyForUser(Context context) {
        String email = UserSession.getEmail(context.getApplicationContext());
        if (email == null || email.isEmpty()) {
            return KEY_SCORE + "guest";
        }
        return KEY_SCORE + safeKey(email);
     }

    public static long getLocalScore(Context context) {
        return prefs(context).getLong(keyForUser(context), 0L);
     }

    public static void addPoints(Context context, int delta) {
        if (delta <= 0) {
```

```
            return;
        }
        String k = keyForUser(context);
        long next = prefs(context).getLong(k, 0L) + delta;
        prefs(context).edit().putLong(k, next).apply();
    }

    public static void getDisplayScore(Context context, Consumer<Long> callback) {
        long score = getLocalScore(context);
        new Handler(Looper.getMainLooper()).post(() -> callback.accept(score));
    }

    public static void clearAll(Context context) {
        prefs(context).edit().clear().apply();
    }
}
```

==========
==========
FILE:
app\src\main\java\com\example\treasurehuntapplication\SettingsFragment.java
==========
==========

```
package com.example.treasurehuntapplication;

import android.os.Bundle;
import android.text.InputType;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.EditText;
import android.widget.ImageButton;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;

import com.google.android.material.appbar.MaterialToolbar;

/**
```

```
 * Editable profile screen: name, location, course, pronouns; name/location sync to
Firestore for leaderboard.
 */
public class SettingsFragment extends Fragment {

    private TextView nameView;
    private TextView locationView;
    private TextView courseView;
    private TextView pronounsView;
    private ImageButton editNameBtn;
    private ImageButton editLocationBtn;
    private ImageButton editCourseBtn;
    private ImageButton editPronounsBtn;

    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, ViewGroup container,
Bundle savedInstanceState) {
        return inflater.inflate(R.layout.fragment_settings, container, false);
    }

    @Override
    public void onViewCreated(@NonNull View view, @Nullable Bundle
savedInstanceState) {
        super.onViewCreated(view, savedInstanceState);

        nameView = view.findViewById(R.id.Profile_Name);
        locationView = view.findViewById(R.id.Profile_Location);
        courseView = view.findViewById(R.id.Profile_Course);
        pronounsView = view.findViewById(R.id.Profile_Pronouns);
        editNameBtn = view.findViewById(R.id.Name_Edit_Button);
        editLocationBtn = view.findViewById(R.id.Location_Edit_button);
        editCourseBtn = view.findViewById(R.id.Course_Edit_Button);
        editPronounsBtn = view.findViewById(R.id.Pronouns_Edit_Button);

        MaterialToolbar toolbar = view.findViewById(R.id.settingsToolbar);
        toolbar.setNavigationOnClickListener(v ->
                requireActivity().getOnBackPressedDispatcher().onBackPressed());

        editNameBtn.setOnClickListener(v -> showTextEditDialog(Field.NAME));
        editLocationBtn.setOnClickListener(v -> showTextEditDialog(Field.LOCATION));
        editCourseBtn.setOnClickListener(v -> showTextEditDialog(Field.COURSE));
        editPronounsBtn.setOnClickListener(v ->
showTextEditDialog(Field.PRONOUNS));

        bindUiFromPrefs();
    }
```

```
    @Override
    public void onResume() {
        super.onResume();
        bindUiFromPrefs();
    }

    private enum Field {
        NAME,
        LOCATION,
        COURSE,
        PRONOUNS
    }

    private void bindUiFromPrefs() {
        bindPrefText(nameView,
ProfilePreferences.getDisplayName(requireContext()));
        bindPrefText(locationView, ProfilePreferences.getLocation(requireContext()));
        bindPrefText(courseView, ProfilePreferences.getCourse(requireContext()));
        bindPrefText(pronounsView,
ProfilePreferences.getPronouns(requireContext()));
    }

    private void bindPrefText(TextView view, String value) {
        view.setText(value == null || value.isEmpty() ?
getString(R.string.settings_not_set) : value);
    }

    private void showTextEditDialog(Field field) {
        int titleRes;
        String current;
        switch (field) {
            case NAME:
                titleRes = R.string.settings_edit_name_title;
                current = ProfilePreferences.getDisplayName(requireContext());
                break;
            case LOCATION:
                titleRes = R.string.settings_edit_location_title;
                current = ProfilePreferences.getLocation(requireContext());
                break;
            case COURSE:
                titleRes = R.string.settings_edit_course_title;
                current = ProfilePreferences.getCourse(requireContext());
                break;
            case PRONOUNS:
            default:
```

```
            titleRes = R.string.settings_edit_pronouns_title;
            current = ProfilePreferences.getPronouns(requireContext());
            break;
        }

        final EditText input = new EditText(requireContext());
        input.setInputType(InputType.TYPE_CLASS_TEXT |
InputType.TYPE_TEXT_FLAG_CAP_SENTENCES);
        input.setText(current);
        input.setSelection(input.getText().length());

        NtuMaterialDialogs.builder(requireContext())
                .setTitle(titleRes)
                .setView(input)
                .setPositiveButton(R.string.settings_ok, (d, w) -> {
                    String v = input.getText() != null ? input.getText().toString().trim() :
"";
                    switch (field) {
                        case NAME:
                            ProfilePreferences.setDisplayName(requireContext(), v);
                            UserProfileFirestoreSync.pushDisplayName(requireContext(), v);
                            break;
                        case LOCATION:
                            ProfilePreferences.setLocation(requireContext(), v);
                            UserProfileFirestoreSync.pushProfileLocation(requireContext(),
v);
                            break;
                        case COURSE:
                            ProfilePreferences.setCourse(requireContext(), v);
                            break;
                        case PRONOUNS:
                            ProfilePreferences.setPronouns(requireContext(), v);
                            break;
                    }
                    bindUiFromPrefs();
                })
                .setNegativeButton(R.string.settings_cancel, (d, w) -> d.cancel())
                .show();
    }
}
```

```
==================================================
====================
```

FILE: app\src\main\java\com\example\treasurehuntapplication\Signup.java

```
================================================================
====================
```

```
package com.example.treasurehuntapplication;

import android.content.Intent;
import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseAuthUserCollisionException;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.firestore.FirebaseFirestore;

import java.util.HashMap;
import java.util.Map;

/**
 * New account registration: Firebase Auth user creation, initial {@code
users/{email}} document
 * in Firestore (email, username placeholder, score 0), then return to {@link Login}.
 * <p>
 * Registration is refused unless the email is a valid {@code @my.ntu.ac.uk}
student address
 * ({@link AuthInputValidator#isAllowedNtuLoginEmail}), same rule as {@link
Login}.
 */
public class Signup extends AppCompatActivity {

    // --- Firebase ---
    private FirebaseFirestore db;
    private FirebaseAuth auth;
    private EditText usernameInput;
    private EditText passwordInput;
    private EditText confirmPasswordInput;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
```

```
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_signup);

        // --- Edge-to-edge insets ---
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v,
windowInsets) -> {
            Insets bars =
windowInsets.getInsets(WindowInsetsCompat.Type.systemBars());
            Insets cutout =
windowInsets.getInsets(WindowInsetsCompat.Type.displayCutout());
            v.setPadding(
                    Math.max(bars.left, cutout.left),
                    Math.max(bars.top, cutout.top),
                    Math.max(bars.right, cutout.right),
                    Math.max(bars.bottom, cutout.bottom));
            return windowInsets;
        });

        // --- Services & fields ---
        db = FirebaseFirestore.getInstance();
        auth = FirebaseAuth.getInstance();

        usernameInput = findViewById(R.id.signup_username);
        passwordInput = findViewById(R.id.signup_password);
        confirmPasswordInput = findViewById(R.id.signup_confirm_password);
        Button signupButton = findViewById(R.id.signup_create);
        Button backToLoginButton = findViewById(R.id.signup_back);

        signupButton.setOnClickListener(v -> createAccount());
        backToLoginButton.setOnClickListener(v -> startActivity(new
Intent(Signup.this, Login.class)));
    }

    /** Validates input, creates Auth user, writes Firestore profile; rolls back Auth if
Firestore fails. */
    private void createAccount() {
        String email =
AuthInputValidator.normalizeEmail(usernameInput.getText().toString());
        String password = passwordInput.getText().toString();
        String confirm = confirmPasswordInput.getText().toString();

        if (email.isEmpty() || password.isEmpty() || confirm.isEmpty()) {
            Toast.makeText(this, R.string.login_empty_fields,
Toast.LENGTH_SHORT).show();
            return;
```

```
        }
        if (!AuthInputValidator.isValidEmail(email)) {
            Toast.makeText(this, R.string.auth_invalid_email,
Toast.LENGTH_SHORT).show();
            return;
        }
        // Align with Login: no Firebase account for Gmail, Outlook, staff-only
ntu.ac.uk, etc.
        if (!AuthInputValidator.isAllowedNtuLoginEmail(email)) {
            Toast.makeText(this, R.string.auth_email_ntu_only,
Toast.LENGTH_LONG).show();
            return;
        }
        if (!AuthInputValidator.isValidPasswordLength(password)) {
            Toast.makeText(this, getString(R.string.auth_password_too_short,
                    AuthInputValidator.MIN_PASSWORD_LENGTH),
Toast.LENGTH_LONG).show();
            return;
        }
        if (!password.equals(confirm)) {
            Toast.makeText(this, R.string.passwords_do_not_match,
Toast.LENGTH_SHORT).show();
            return;
        }

        auth.createUserWithEmailAndPassword(email, password)
                .addOnSuccessListener(authResult -> {
                    FirebaseUser user = authResult.getUser();
                    String rawId = user != null && user.getEmail() != null ?
user.getEmail() : email;
                    String docId = AuthInputValidator.normalizeEmail(rawId);
                    Map<String, Object> profile = new HashMap<>();
                    if (user != null) {
                        profile.put("uid", user.getUid());
                    }
                    profile.put("email", docId);
                    profile.put("username", docId);
                    profile.put("score", 0L);

                    db.collection("users").document(docId).set(profile)
                            .addOnSuccessListener(aVoid -> {
                                Toast.makeText(Signup.this, R.string.account_created,
Toast.LENGTH_SHORT).show();
                                goToLogin();
                            })
                            .addOnFailureListener(e -> {
```

```
                        if (user != null) {
                            user.delete();
                        }
                        Toast.makeText(Signup.this,
R.string.auth_firestore_profile_failed,
                                Toast.LENGTH_LONG).show();
                    });
            })
            .addOnFailureListener(e -> {
                if (e instanceof FirebaseAuthUserCollisionException) {
                    Toast.makeText(Signup.this, R.string.auth_signup_failed,
Toast.LENGTH_LONG).show();
                } else {
                    Toast.makeText(Signup.this, R.string.auth_signup_failed,
Toast.LENGTH_LONG).show();
                }
            });
    }

    /** Returns to login without stacking duplicate activities. */
    private void goToLogin() {
        Intent i = new Intent(Signup.this, Login.class);
        i.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK |
Intent.FLAG_ACTIVITY_CLEAR_TOP);
        startActivity(i);
        finish();
    }
}
```

```
==================================================
====================
FILE:
app\src\main\java\com\example\treasurehuntapplication\SupportFragment.java
==================================================
====================

package com.example.treasurehuntapplication;

import android.os.Bundle;
 import android.view.LayoutInflater;
 import android.view.View;
 import android.view.ViewGroup;
 import android.widget.ArrayAdapter;
 import android.widget.Spinner;
```

```
 import android.widget.Toast;

import androidx.annotation.NonNull;
 import androidx.annotation.Nullable;
 import androidx.fragment.app.Fragment;

import com.google.android.material.button.MaterialButton;
 import com.google.android.material.textfield.TextInputEditText;
 import com.google.firebase.auth.FirebaseAuth;
 import com.google.firebase.firestore.FirebaseFirestore;

import java.util.HashMap;
 import java.util.Map;

/**
  * In-app support form: submits category + message to Firestore {@code
support_reports}
  * when the user is signed in with Firebase Auth.
  */
 public class SupportFragment extends Fragment {

    @Nullable
     @Override
     public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup
container,
             @Nullable Bundle savedInstanceState) {
         return inflater.inflate(R.layout.fragment_support, container, false);
     }

    @Override
     public void onViewCreated(@NonNull View view, @Nullable Bundle
savedInstanceState) {
         super.onViewCreated(view, savedInstanceState);

        // Ensure the form scroll layer draws above the gradient header (matches
design overlap).
         View scroll = view.findViewById(R.id.supportScroll);
         ViewGroup root = (ViewGroup) view;
         Runnable liftScrollLayer = () -> root.bringChildToFront(scroll);
         liftScrollLayer.run();
         scroll.post(liftScrollLayer);

        Spinner spinner = view.findViewById(R.id.supportCategorySpinner);
         ArrayAdapter<CharSequence> adapter =
ArrayAdapter.createFromResource(requireContext(),
                 R.array.support_categories, R.layout.support_spinner_item);
```

```
adapter.setDropDownViewResource(R.layout.support_spinner_dropdown_item);
        spinner.setAdapter(adapter);

       TextInputEditText messageInput =
view.findViewById(R.id.supportMessageInput);
        MaterialButton submit = view.findViewById(R.id.supportSubmitButton);

       submit.setOnClickListener(v -> {
           CharSequence msg = messageInput.getText();
           String body = msg != null ? msg.toString().trim() : "";
           if (body.isEmpty()) {
               Toast.makeText(requireContext(), R.string.support_empty_message,
Toast.LENGTH_SHORT).show();
               return;
           }
           if (FirebaseAuth.getInstance().getCurrentUser() == null) {
               Toast.makeText(requireContext(), R.string.support_failed,
Toast.LENGTH_LONG).show();
               return;
           }
           String category = spinner.getSelectedItem() != null
                   ? spinner.getSelectedItem().toString()
                   : "";

          Map<String, Object> report = new HashMap<>();
           report.put("category", category);
           report.put("message", body);
           report.put("userEmail", UserSession.getEmail(requireContext()));
           report.put("createdAt",
com.google.firebase.firestore.FieldValue.serverTimestamp());

          FirebaseFirestore.getInstance()
                   .collection("support_reports")
                   .add(report)
                   .addOnSuccessListener(ref -> {
                       Toast.makeText(requireContext(), R.string.support_sent,
Toast.LENGTH_LONG).show();
                       messageInput.setText("");
                   })
                   .addOnFailureListener(e ->
                           Toast.makeText(requireContext(), R.string.support_failed,
Toast.LENGTH_LONG).show());
       });
    }
```

```
}
```

```
================================================================================
FILE: app\src\main\java\com\example\treasurehuntapplication\Treasure.java
================================================================================

package com.example.treasurehuntapplication;

/**
 * Hunt definition (clue, GPS, rarity, points). Whether the player has collected it is stored
 * per-user in {@link TreasureManager}, not on this object.
 */
public class Treasure {

    /** Hunt difficulty tier (drives UI colour and point value in catalogue). */
    public enum Rarity {
        COMMON,
        RARE,
        LEGENDARY
    }

    // --- Immutable hunt definition ---
    private final int id;
    private final String title;
    private final String clue;
    private final String hint;
    private final double latitude;
    private final double longitude;
    private final Rarity rarity;
    private final int points;

    /** Full constructor used by {@link TreasureManager} static catalogue. */
    public Treasure(int id, String title, String clue, String hint,
                    double latitude, double longitude, Rarity rarity, int points) {
        this.id = id;
        this.title = title;
        this.clue = clue;
        this.hint = hint;
        this.latitude = latitude;
        this.longitude = longitude;
        this.rarity = rarity;
```

```
        this.points = points;
    }

    // --- Getters ---

    public int getId() {
        return id;
    }

    public String getTitle() {
        return title;
    }

    public String getClue() {
        return clue;
    }

    public String getHint() {
        return hint;
    }

    public double getLatitude() {
        return latitude;
    }

    public double getLongitude() {
        return longitude;
    }

    public Rarity getRarity() {
        return rarity;
    }

    public int getPoints() {
        return points;
    }
}
```

==========
==========
FILE:
app\src\main\java\com\example\treasurehuntapplication\TreasureAchievements.java

```
================================================================
====================

package com.example.treasurehuntapplication;

/** Treasure-count achievement definitions for the stats hex grid. */
 public final class TreasureAchievements {

    public static final class Badge {
        public final int drawableRes;
        public final int requiredFound;
        public final int labelRes;
        public final int detailRes;

       public Badge(int drawableRes, int requiredFound, int labelRes, int detailRes) {
            this.drawableRes = drawableRes;
            this.requiredFound = requiredFound;
            this.labelRes = labelRes;
            this.detailRes = detailRes;
        }
    }

    private TreasureAchievements() {
    }

    public static Badge[] getTreasureBadges() {
        return new Badge[]{
                new Badge(R.drawable.treasure_1, 1, R.string.achievement_treasure_1,
                        R.string.achievement_treasure_1_detail),
                new Badge(R.drawable.treasure_3, 3, R.string.achievement_treasure_3,
                        R.string.achievement_treasure_3_detail),
                new Badge(R.drawable.treasure_5, 5, R.string.achievement_treasure_5,
                        R.string.achievement_treasure_5_detail),
        };
    }
 }




================================================================
====================
FILE:
app\src\main\java\com\example\treasurehuntapplication\TreasureFragment.java
================================================================
====================
```

```
package com.example.treasurehuntapplication;

import android.content.res.ColorStateList;
import android.graphics.Color;
import android.graphics.Typeface;
import android.os.Bundle;
import android.view.Gravity;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.FrameLayout;
import android.widget.ImageView;
import android.widget.LinearLayout;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AlertDialog;
import androidx.core.content.ContextCompat;
import androidx.fragment.app.Fragment;

import com.google.android.material.card.MaterialCardView;

import com.example.treasurehuntapplication.Treasure.Rarity;

/**
 * Browse treasures by rarity: tap a tile for clue + hint, accept to set {@link
ActiveTreasureManager}
 * and switch to the Map tab in {@link navigation}.
 */
public class TreasureFragment extends Fragment {

    public TreasureFragment() {
    }

    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, ViewGroup container,
                             Bundle savedInstanceState) {
        View view = inflater.inflate(R.layout.fragment_treasure, container, false);
        populateTreasures(view);
        return view;
    }

    @Override
```

```
    public void onResume() {
        super.onResume();
        if (getView() != null) {
            populateTreasures(getView());
        }
    }

    /** Rebuilds horizontal lists per rarity and updates section headers with counts.
*/
    private void populateTreasures(View view) {
        LinearLayout commonContainer = view.findViewById(R.id.commonContainer);
        LinearLayout rareContainer = view.findViewById(R.id.rareContainer);
        LinearLayout legendaryContainer =
view.findViewById(R.id.legendaryContainer);

        TextView commonHeader = view.findViewById(R.id.commonCount);
        TextView rareHeader = view.findViewById(R.id.rareCount);
        TextView legendaryHeader = view.findViewById(R.id.legendaryCount);

        commonHeader.setText(String.format("Common â€” %d treasures",
TreasureManager.countByRarity(Rarity.COMMON)));
        rareHeader.setText(String.format("Rare â€” %d treasures",
TreasureManager.countByRarity(Rarity.RARE)));
        legendaryHeader.setText(String.format("Legendary â€” %d treasure",
TreasureManager.countByRarity(Rarity.LEGENDARY)));

        commonContainer.removeAllViews();
        rareContainer.removeAllViews();
        legendaryContainer.removeAllViews();

        for (Treasure t : TreasureManager.getByRarity(Rarity.COMMON)) {
            addTreasureTile(commonContainer, t);
        }
        for (Treasure t : TreasureManager.getByRarity(Rarity.RARE)) {
            addTreasureTile(rareContainer, t);
        }
        for (Treasure t : TreasureManager.getByRarity(Rarity.LEGENDARY)) {
            addTreasureTile(legendaryContainer, t);
        }
    }

    /** One scroll tile: coloured card, tap opens popup if not yet collected. */
    private void addTreasureTile(LinearLayout container, Treasure treasure) {
        MaterialCardView wrapper = new MaterialCardView(requireContext());
        LinearLayout.LayoutParams wrapParams =
                new LinearLayout.LayoutParams(500, 500);
```

```
        wrapParams.setMargins(0, 0, 20, 0);
        wrapper.setLayoutParams(wrapParams);
        float density = getResources().getDisplayMetrics().density;
        int cornerPx = Math.round(12 * density);
        int strokePx = Math.max(1, Math.round(1 * density));
        wrapper.setRadius(cornerPx);
        wrapper.setCardElevation(0f);
        wrapper.setStrokeWidth(strokePx);
        wrapper.setStrokeColor(ColorStateList.valueOf(
                ContextCompat.getColor(requireContext(),
R.color.ntu_leaderboard_row_stroke)));
        wrapper.setCardBackgroundColor(tileColor(treasure.getRarity()));
        wrapper.setPreventCornerOverlap(false);
        wrapper.setUseCompatPadding(false);

        ImageView img = new ImageView(requireContext());
        FrameLayout.LayoutParams imgParams = new FrameLayout.LayoutParams(
                FrameLayout.LayoutParams.MATCH_PARENT,
                FrameLayout.LayoutParams.MATCH_PARENT);
        img.setLayoutParams(imgParams);
        boolean found = TreasureManager.isFound(requireContext(),
treasure.getId());
        // Do not use setEnabled(false) when collected â€” that blocks long-press for
"reset for testing".
        img.setAlpha(found ? 0.88f : 1f);

        if (!found) {
            img.setOnClickListener(v -> showTreasurePopup(treasure));
        }

        View.OnLongClickListener resetCollectedListener = v -> {
            if (!TreasureManager.isFound(requireContext(), treasure.getId())) {
                return false;
            }
            NtuMaterialDialogs.builder(requireContext())
                    .setTitle(R.string.treasure_reset_collected_title)
                    .setMessage(R.string.treasure_reset_collected_message)
                    .setPositiveButton(R.string.treasure_reset_collected_confirm, (d, w) -
> {
                        TreasureManager.unmarkFound(requireContext(),
treasure.getId());
                        View root = getView();
                        if (root != null) {
                            populateTreasures(root);
                        }
                    })
```

```
                .setNegativeButton(R.string.settings_cancel, null)
                .show();
            return true;
        };
        img.setLongClickable(true);
        img.setHapticFeedbackEnabled(true);
        img.setOnLongClickListener(resetCollectedListener);

        wrapper.addView(img);

        if (found) {
            TextView badge = new TextView(requireContext());
            badge.setText(R.string.treasure_collected_badge);
            badge.setTextColor(Color.WHITE);
            badge.setTypeface(null, Typeface.BOLD);
            badge.setTextSize(14f);
            badge.setGravity(Gravity.CENTER);
            badge.setBackgroundColor(0xCC000000);
            FrameLayout.LayoutParams badgeParams = new
FrameLayout.LayoutParams(
                    FrameLayout.LayoutParams.MATCH_PARENT,
                    FrameLayout.LayoutParams.WRAP_CONTENT);
            badgeParams.gravity = Gravity.BOTTOM;
            badge.setPadding(8, 10, 8, 10);
            badge.setLongClickable(true);
            badge.setHapticFeedbackEnabled(true);
            badge.setOnLongClickListener(resetCollectedListener);
            wrapper.addView(badge, badgeParams);
        }

        container.addView(wrapper);
    }

    /** Background tint by rarity (matches Stats screen palette). */
    private static int tileColor(Rarity rarity) {
        switch (rarity) {
            case COMMON:
                return Color.rgb(76, 175, 80);
            case RARE:
                return Color.rgb(33, 150, 243);
            case LEGENDARY:
                return Color.rgb(255, 193, 7);
            default:
                return Color.GRAY;
        }
    }
```

```
    /** Clue dialog; Accept stores active treasure id and navigates to map. */
    private void showTreasurePopup(Treasure treasure) {

        LayoutInflater inflater = getLayoutInflater();
        View dialogView = inflater.inflate(R.layout.item_hint, null);

        TextView tvTitle = dialogView.findViewById(R.id.tvTitle);
        TextView tvHint = dialogView.findViewById(R.id.tvHint);
        Button btnAccept = dialogView.findViewById(R.id.btnAccept);

        tvTitle.setText(treasure.getTitle());
        tvHint.setText(treasure.getClue() + "\n\nHint: " + treasure.getHint());

        AlertDialog dialog = NtuMaterialDialogs.builder(requireContext())
                .setView(dialogView)
                .create();
        NtuMaterialDialogs.applyCustomContentWindow(dialog, 0.7f);
        dialog.show();

        btnAccept.setOnClickListener(v -> {
            ActiveTreasureManager.setActiveTreasure(requireContext(),
treasure.getId());
            dialog.dismiss();
            if (getActivity() instanceof navigation) {
                ((navigation) getActivity()).showMapTab();
            }
        });
    }
}
```

```
==========================================================
====================
FILE:
app\src\main\java\com\example\treasurehuntapplication\TreasureManager.java
==========================================================
====================

package com.example.treasurehuntapplication;

import android.content.Context;
import android.content.SharedPreferences;
```

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * Single source of truth for hunt definitions (clues, GPS, rarity, points).
 * Collected state is stored per signed-in user in {@link SharedPreferences}.
 */
public final class TreasureManager {

    // --- Preferences: which treasure ids are "found" per user email ---
    private static final String PREF = "treasure_found_state";
    private static final String KEY_FOUND_SET_PREFIX = "found_ids_";

    private static final List<Treasure> TREASURES = new ArrayList<>();

    // --- Static hunt catalogue (id, copy, GPS, rarity, points) ---
    static {
        TREASURES.add(new Treasure(
                1,
                "Common",
                "I hold knowledge but make no sound. Find me where silence is loud.",
                "Look for water nearby, close to a spot where students play.",
                52.912014622528005, -1.1867476981535494,
                Treasure.Rarity.COMMON,
                10
        ));
        TREASURES.add(new Treasure(
                2,
                "Common",
                "Bats may swing and cheers may sound. Find the field where cricketâ€™s
found.",
                "Stay on the grass and search near the edge of the playing area.",
                52.9143221975777, -1.1846936807536121,
                Treasure.Rarity.COMMON,
                10
        ));
        TREASURES.add(new Treasure(
                3,
                "Rare",
                "Sticks may clash and goals are chased. Find the turf where hockeyâ€™s
placed.",
                "Look along the lines where players wait to sub on and cheer.",
                52.91253722913912, -1.1888147758766068,
```

```
            Treasure.Rarity.RARE,
            25
        ));
        TREASURES.add(new Treasure(
            4,
            "Rare",
            "Ideas rise in glass and steel. Find the STEM building where futures feel
real.",
            "Search near the busiest entrance where people come and go.",
            52.911170433353334, -1.1845803402665962,
            Treasure.Rarity.RARE,
            25
        ));
        TREASURES.add(new Treasure(
            5,
            "Rare",
            "Curious minds and lessons meet. Find the School of Science &
Technology seat.",
            "Try the main doorsâ€”where the building first welcomes you in.",
            52.91055489051997, -1.1866600260573863,
            Treasure.Rarity.RARE,
            25
        ));
        TREASURES.add(new Treasure(
            6,
            "Legendary",
            "Voices gather, history stays. Seek the old stone by Village Hallâ€™s
ways.",
            "Find the old stone close to a place where people gather for events.",
            52.90843613936876, -1.1870923367236603,
            Treasure.Rarity.LEGENDARY,
            100
        ));
        TREASURES.add(new Treasure(
            7,
            "Legendary",
            "Mirrors, chairs, and careful handsâ€”find the beautification parlour
where style expands.",
            "Stand where the parlour meets the pavement; the treasure waits near
the beauticianâ€™s door.",
            52.95562325166844, -1.1567528670292193,
            Treasure.Rarity.LEGENDARY,
            100
        ));
    }

    private TreasureManager() {
```

```
    }

    // --- Preference key helpers (scoped by UserSession email) ---

    private static SharedPreferences prefs(Context context) {
        return context.getApplicationContext().getSharedPreferences(PREF,
Context.MODE_PRIVATE);
    }

    private static String safeKey(String raw) {
        return raw.trim().toLowerCase().replaceAll("[^a-z0-9]+", "_");
    }

    private static String foundSetKey(Context context) {
        String email = UserSession.getEmail(context.getApplicationContext());
        if (email == null || email.isEmpty()) {
            return KEY_FOUND_SET_PREFIX + "guest";
        }
        return KEY_FOUND_SET_PREFIX + safeKey(email);
    }

    // --- Per-user collection state ---

    public static boolean isFound(Context context, int treasureId) {
        Set<String> set = prefs(context).getStringSet(foundSetKey(context),
Collections.emptySet());
        return set.contains(String.valueOf(treasureId));
    }

    public static void markFound(Context context, int treasureId) {
        Set<String> next = new
HashSet<>(prefs(context).getStringSet(foundSetKey(context),
Collections.emptySet()));
        next.add(String.valueOf(treasureId));
        prefs(context).edit().putStringSet(foundSetKey(context), next).apply();
    }

    /** Removes one treasure from the collected set (e.g. debug / retesting a
location). */
    public static void unmarkFound(Context context, int treasureId) {
        Set<String> next = new
HashSet<>(prefs(context).getStringSet(foundSetKey(context),
Collections.emptySet()));
        next.remove(String.valueOf(treasureId));
        prefs(context).edit().putStringSet(foundSetKey(context), next).apply();
    }
```

```
    public static int getFoundCount(Context context) {
        return prefs(context).getStringSet(foundSetKey(context),
Collections.emptySet()).size();
    }

    /** Ordered ids of treasures this user has collected (for Firestore sync). */
    public static List<Long> getFoundTreasureIdsForSync(Context context) {
        Set<String> raw = prefs(context).getStringSet(foundSetKey(context),
Collections.emptySet());
        List<Long> out = new ArrayList<>();
        for (String s : raw) {
            try {
                out.add(Long.parseLong(s));
            } catch (NumberFormatException ignored) {
                // skip
            }
        }
        Collections.sort(out);
        return out;
    }

    // --- Read-only access to catalogue ---

    public static List<Treasure> getAll() {
        return Collections.unmodifiableList(TREASURES);
    }

    public static List<Treasure> getByRarity(Treasure.Rarity rarity) {
        List<Treasure> out = new ArrayList<>();
        for (Treasure t : TREASURES) {
            if (t.getRarity() == rarity) {
                out.add(t);
            }
        }
        return out;
    }

    public static Treasure getById(int id) {
        for (Treasure t : TREASURES) {
            if (t.getId() == id) {
                return t;
            }
        }
        return null;
    }
```

```
    public static Treasure getNextTreasure(Context context) {
        for (Treasure t : TREASURES) {
            if (!isFound(context, t.getId())) {
                return t;
            }
        }
        return null;
    }

    public static int getTotalCount() {
        return TREASURES.size();
    }

    public static int countByRarity(Treasure.Rarity rarity) {
        int n = 0;
        for (Treasure t : TREASURES) {
            if (t.getRarity() == rarity) {
                n++;
            }
        }
        return n;
    }

    // --- Maintenance (device-wide) ---

    /** Clears collected-treasure flags for every account on this device. */
    public static void clearAllCollectedState(Context context) {
        prefs(context).edit().clear().apply();
    }
}
```

```
================================================================================
FILE: app\src\main\java\com\example\treasurehuntapplication\TutorialPopup.java
================================================================================

package com.example.treasurehuntapplication;

import android.app.Activity;
 import android.content.Context;
 import android.content.SharedPreferences;
```

```
 import android.view.LayoutInflater;
 import android.view.View;

import androidx.appcompat.app.AlertDialog;

import com.google.firebase.auth.FirebaseAuth;
 import com.google.firebase.auth.FirebaseUser;

/** One-time â€œhow to playâ€ dialog per signed-in account (Firebase UID when
available). */
 public final class TutorialPopup {

    private static final String PREF = "tutorial_popup";
     private static final String KEY_PREFIX = "seen_";

    private TutorialPopup() {
     }

    private static String safeKey(String raw) {
        return raw.trim().toLowerCase().replaceAll("[^a-z0-9]+", "_");
     }

    private static SharedPreferences prefs(Context context) {
        return context.getApplicationContext().getSharedPreferences(PREF,
Context.MODE_PRIVATE);
     }

    /** Stable id: Firebase {@code uid}, else normalized email, else {@code guest}.
*/
     private static String tutorialAccountKey(Context context) {
        FirebaseUser u = FirebaseAuth.getInstance().getCurrentUser();
        if (u != null) {
            String uid = u.getUid();
            if (uid != null && !uid.isEmpty()) {
                return uid;
            }
        }
        String email = UserSession.getEmail(context);
        if (email != null && !email.isEmpty()) {
            return safeKey(email);
        }
        return "guest";
     }

    public static void showIfFirstTime(Activity activity) {
```

```
        String key = KEY_PREFIX + tutorialAccountKey(activity);
        if (prefs(activity).getBoolean(key, false)) {
            return;
        }

        View content = LayoutInflater.from(activity).inflate(R.layout.dialog_tutorial,
null);
        AlertDialog dialog = NtuMaterialDialogs.builder(activity)
                .setView(content)
                .setCancelable(false)
                .create();
        NtuMaterialDialogs.applyCustomContentWindow(dialog, 0.65f);

        content.findViewById(R.id.tutorialOk).setOnClickListener(v -> {
            prefs(activity).edit().putBoolean(key, true).apply();
            dialog.dismiss();
        });

        dialog.show();
    }
 }
```

```
==========================================================
====================
FILE:
app\src\main\java\com\example\treasurehuntapplication\UserGameStateFirestoreSy
nc.java
==========================================================
====================
```

```
package com.example.treasurehuntapplication;

import android.content.Context;

import com.google.firebase.auth.FirebaseAuth;
 import com.google.firebase.auth.FirebaseUser;
 import com.google.firebase.firestore.FirebaseFirestore;
 import com.google.firebase.firestore.SetOptions;

import java.util.HashMap;
 import java.util.List;
 import java.util.Map;
```

```
/** Pushes score, walk distance, and found-treasure ids to {@code users/{email}}.
*/
 public final class UserGameStateFirestoreSync {

    private UserGameStateFirestoreSync() {
     }

    private static String documentId(Context context) {
        return FirestoreUserDoc.idForCurrentUser(context.getApplicationContext());
     }

    public static void pushGameStateToFirestore(Context context) {
        FirebaseUser u = FirebaseAuth.getInstance().getCurrentUser();
        if (u == null) {
            return;
        }
        String docId = documentId(context);
        if (docId == null || docId.isEmpty()) {
            return;
        }
        List<Long> found = TreasureManager.getFoundTreasureIdsForSync(context);
        Map<String, Object> data = new HashMap<>();
        data.put("uid", u.getUid());
        data.put("score", ScoreRepository.getLocalScore(context));
        data.put("totalWalkMeters", WalkDistanceTracker.getTotalMeters(context));
        data.put("foundTreasureIds", found);

       String campusLine =
ProfilePreferences.getLocation(context.getApplicationContext());
        if (campusLine != null && !campusLine.trim().isEmpty()) {
            data.put("profileLocation", campusLine.trim());
        }

       FirebaseFirestore.getInstance()
                .collection("users")
                .document(docId)
                .set(data, SetOptions.merge());
     }

    public static void wipeRemoteGameFieldsForCurrentUser(Context context) {
        FirebaseUser u = FirebaseAuth.getInstance().getCurrentUser();
        if (u == null) {
            return;
        }
        String docId = FirestoreUserDoc.idForFirebaseUser(u);
        if (docId == null || docId.isEmpty()) {
```

```
            return;
        }
        Map<String, Object> data = new HashMap<>();
        data.put("score", 0L);
        data.put("totalWalkMeters", 0L);
        data.put("foundTreasureIds", java.util.Collections.emptyList());

       FirebaseFirestore.getInstance()
                .collection("users")
                .document(docId)
                .set(data, SetOptions.merge());
    }
 }
```

```
================================================================
====================
FILE:
app\src\main\java\com\example\treasurehuntapplication\UserProfileFirestoreSync.ja
va
================================================================
====================
```

```
package com.example.treasurehuntapplication;

import android.content.Context;
 import android.content.SharedPreferences;
 import android.widget.Toast;

import com.google.firebase.auth.FirebaseAuth;
 import com.google.firebase.auth.FirebaseUser;
 import com.google.firebase.firestore.FirebaseFirestore;
 import com.google.firebase.firestore.SetOptions;

import java.util.HashMap;
 import java.util.Map;

/** Syncs display name with {@code users/{email}.username} and hydrates from
Firestore once per account. */
 public final class UserProfileFirestoreSync {

    private static final String PREF = "profile_fs_hydration";
     private static final String KEY_HYDRATED = "hydrated_";
```

```
    private UserProfileFirestoreSync() {
     }

    private static SharedPreferences prefs(Context context) {
        return context.getApplicationContext().getSharedPreferences(PREF,
Context.MODE_PRIVATE);
     }

    private static String safeKey(String raw) {
        return raw.trim().toLowerCase().replaceAll("[^a-z0-9]+", "_");
     }

    public static void clearAccountHydrationMarker(Context context) {
        String email = UserSession.getEmail(context.getApplicationContext());
        if (email != null && !email.isEmpty()) {
            prefs(context).edit().remove(KEY_HYDRATED + safeKey(email)).apply();
        }
     }

    public static void hydrateDisplayNameForCurrentAccount(Context context) {
        Context app = context.getApplicationContext();
        String docId = FirestoreUserDoc.idForCurrentUser(app);
        if (docId == null || docId.isEmpty()) {
            return;
        }
        FirebaseUser u = FirebaseAuth.getInstance().getCurrentUser();
        if (u == null || u.getEmail() == null || u.getEmail().isEmpty()) {
            return;
        }
        String key = KEY_HYDRATED + safeKey(docId);
        if (prefs(app).getBoolean(key, false)) {
            return;
        }
        FirebaseFirestore.getInstance()
                .collection("users")
                .document(docId)
                .get()
                .addOnSuccessListener(doc -> {
                    if (doc.exists()) {
                        String un = doc.getString("username");
                        String local = ProfilePreferences.getDisplayName(app);
                        if (un != null && !un.trim().isEmpty()
                                && (local == null || local.trim().isEmpty())) {
                            ProfilePreferences.setDisplayName(app, un.trim());
                        }
                    }
```

```
                prefs(app).edit().putBoolean(key, true).apply();
            })
            .addOnFailureListener(e -> prefs(app).edit().putBoolean(key,
true).apply());
    }

   public static void pushDisplayName(Context context, String displayName) {
        Context app = context.getApplicationContext();
        String docId = FirestoreUserDoc.idForCurrentUser(app);
        if (docId == null || docId.isEmpty()) {
            return;
        }
        FirebaseUser u = FirebaseAuth.getInstance().getCurrentUser();
        if (u == null) {
            Toast.makeText(app, R.string.profile_sync_needs_firebase_auth,
Toast.LENGTH_LONG).show();
            return;
        }
        Map<String, Object> data = new HashMap<>();
        data.put("uid", u.getUid());
        data.put("username", displayName != null ? displayName : "");
        FirebaseFirestore.getInstance()
                .collection("users")
                .document(docId)
                .set(data, SetOptions.merge())
                .addOnSuccessListener(v ->
                        Toast.makeText(app, R.string.profile_sync_leaderboard_ok,
Toast.LENGTH_SHORT).show())
                .addOnFailureListener(e ->
                        Toast.makeText(app, R.string.profile_sync_leaderboard_failed,
Toast.LENGTH_LONG).show());
    }

   /** Pushes campus/location line for leaderboard subtitle ({@code profileLocation}
on {@code users/{email}}). */
    public static void pushProfileLocation(Context context, String location) {
        Context app = context.getApplicationContext();
        String docId = FirestoreUserDoc.idForCurrentUser(app);
        if (docId == null || docId.isEmpty()) {
            return;
        }
        FirebaseUser u = FirebaseAuth.getInstance().getCurrentUser();
        if (u == null) {
            return;
        }
        Map<String, Object> data = new HashMap<>();
```

```
        data.put("uid", u.getUid());
        data.put("profileLocation", location != null ? location : "");
        FirebaseFirestore.getInstance()
                .collection("users")
                .document(docId)
                .set(data, SetOptions.merge());
    }
}
```

```
================================================================================
FILE: app\src\main\java\com\example\treasurehuntapplication\UserSession.java
================================================================================

package com.example.treasurehuntapplication;

import android.content.Context;
 import android.content.SharedPreferences;

/** Persists the signed-in user email for offline flows and for scoping local
preferences. */
 public final class UserSession {

    private static final String PREF = "user_session";
     private static final String KEY_EMAIL = "email";

    private UserSession() {
     }

    private static SharedPreferences prefs(Context context) {
        return context.getApplicationContext().getSharedPreferences(PREF,
Context.MODE_PRIVATE);
     }

    public static void signIn(Context context, String email) {
        if (email == null || email.trim().isEmpty()) {
            signOut(context);
            return;
        }
        prefs(context).edit().putString(KEY_EMAIL,
email.trim().toLowerCase()).apply();
     }
```

```
    public static void signOut(Context context) {
        prefs(context).edit().remove(KEY_EMAIL).apply();
    }

    public static String getEmail(Context context) {
        String e = prefs(context).getString(KEY_EMAIL, null);
        if (e == null || e.isEmpty()) {
            return null;
        }
        return e;
    }

    public static boolean isSignedIn(Context context) {
        return getEmail(context) != null;
    }
}
```

```
================================================================
====================
FILE:
app\src\main\java\com\example\treasurehuntapplication\WalkAchievements.java
================================================================
====================
```

```
package com.example.treasurehuntapplication;

/** Walk-distance achievement definitions for the stats hex grid. */
public final class WalkAchievements {

    public static final class Badge {
        public final int drawableRes;
        public final int requiredMeters;
        public final int labelRes;
        /** Long description shown when the user taps the badge. */
        public final int detailRes;

        public Badge(int drawableRes, int requiredMeters, int labelRes, int detailRes) {
            this.drawableRes = drawableRes;
            this.requiredMeters = requiredMeters;
            this.labelRes = labelRes;
            this.detailRes = detailRes;
        }
```

```
    }

    private WalkAchievements() {
    }

    public static Badge[] getWalkBadges() {
        return new Badge[]{
                new Badge(R.drawable.achievement_10m, 10,
R.string.achievement_walk_10m,
                        R.string.achievement_walk_10m_detail),
                new Badge(R.drawable.achievement_50m, 100,
R.string.achievement_walk_100m,
                        R.string.achievement_walk_100m_detail),
                new Badge(R.drawable.achievement_100m, 500,
R.string.achievement_walk_500m,
                        R.string.achievement_walk_500m_detail),
        };
    }
}
```

==========
==========
FILE:
app\src\main\java\com\example\treasurehuntapplication\WalkDistanceTracker.java
==========
==========

```
package com.example.treasurehuntapplication;

import android.content.Context;
import android.content.SharedPreferences;
import android.location.Location;

/**
 * Accumulates approximate walk distance from GPS updates (per signed-in user on
this device).
 */
public final class WalkDistanceTracker {

    private static final String PREF = "walk_distance";
    private static final String KEY_METERS = "meters_";
    private static final String KEY_LAT = "lat_";
    private static final String KEY_LON = "lon_";
```

```
    private static final float MAX_SEGMENT_METERS = 500f;
    private static final String KEY_TOAST_PREFIX = "walk_milestone_toast_";
    private static final int[] WALK_MILESTONES = {10, 100, 500};

   private WalkDistanceTracker() {
    }

   private static SharedPreferences prefs(Context context) {
       return context.getApplicationContext().getSharedPreferences(PREF,
Context.MODE_PRIVATE);
    }

   private static String safeKey(String raw) {
       return raw.trim().toLowerCase().replaceAll("[^a-z0-9]+", "_");
    }

   private static String suffix(Context context) {
       String email = UserSession.getEmail(context.getApplicationContext());
       if (email == null || email.isEmpty()) {
           return "guest";
       }
       return safeKey(email);
    }

   public static void recordLocation(Context context, Location loc) {
       if (loc == null) {
           return;
       }
       SharedPreferences p = prefs(context);
       String s = suffix(context);
       float lastLat = p.getFloat(KEY_LAT + s, Float.NaN);
       float lastLon = p.getFloat(KEY_LON + s, Float.NaN);
       long meters = p.getLong(KEY_METERS + s, 0L);

      if (!Float.isNaN(lastLat) && !Float.isNaN(lastLon)) {
           float[] d = new float[1];
           Location.distanceBetween(lastLat, lastLon, loc.getLatitude(),
loc.getLongitude(), d);
           if (d[0] > 0f && d[0] <= MAX_SEGMENT_METERS) {
               meters += (long) d[0];
           }
       }

       p.edit()
               .putLong(KEY_METERS + s, meters)
               .putFloat(KEY_LAT + s, (float) loc.getLatitude())
```

```
                .putFloat(KEY_LON + s, (float) loc.getLongitude())
                .apply();
    }

    public static long getTotalMeters(Context context) {
        return prefs(context).getLong(KEY_METERS + suffix(context), 0L);
    }

    /**
     * Records GPS movement, then returns the smallest walk milestone (10 / 100 /
500 m) that was
     * just crossed for the first time â€” used for a one-shot achievement toast on
the map.
     */
    public static int onLocation(Context context, Location loc) {
        recordLocation(context, loc);
        long total = getTotalMeters(context);
        String s = suffix(context);
        SharedPreferences p = prefs(context);
        for (int m : WALK_MILESTONES) {
            String key = KEY_TOAST_PREFIX + m + "_" + s;
            if (total >= m && !p.getBoolean(key, false)) {
                p.edit().putBoolean(key, true).apply();
                return m;
            }
        }
        return 0;
    }

    public static void clearAll(Context context) {
        prefs(context).edit().clear().apply();
    }
}
```

==========
==========
FILE: app\src\test\java\com\example\treasurehuntapplication\ExampleUnitTest.java
==========
==========

```
package com.example.treasurehuntapplication;

import org.junit.Test;
```

```
import static org.junit.Assert.*;

/**
 * Example local unit test, which will execute on the development machine (host).
 *
 * @see <a href="http://d.android.com/tools/testing">Testing documentation</a>
 */
public class ExampleUnitTest {
    @Test
    public void addition_isCorrect() {
        assertEquals(4, 2 + 2);
    }
}
```